MODULE-1

Practical No. 1

Aim:- Introduction to AI and Intelligence Agents.

a) Design a simple Rational Agent by defining PEAS for Vacuum Cleaner Environment and Autonomous Taxi. Classify Environments (Fully or partially observable, deterministic or stockist, and Episodic/Sequential).Implement a simple table driven agent in Python.

PEAS

1. Vacuum Cleaner

a. Performance Major :- To clean the Room, Minimum Time, Minimum Hour usage.
b. Environments: - Floor, Room or Surroundings.
c. Actuators: - Wheels, Vacuum Motor, Move Left, Move Right, Sucks.
d. Sensors: - Dust detector, Wall detector, Camera.

2. Autonomous Taxi

a. Performance Major: - To reach the destination safely.
b. Environments: - Roads, Traffic, Vehicles.
c. Actuators: - Accelerators, Brakes, Steering wheels.
d. Sensors: - GPS, Camera.

Environment Classification.

1. Vacuum Cleaner

Property	Type
1. Fully Observable	Fully Observable
2. Deterministic	Deterministic
3. Nature	Sequential

2. Autonomous Taxi

Property	Type
1. Fully Observable	Partially Observable
2. Deterministic	Stockist
3. Nature	Sequential

Table Driven Agent

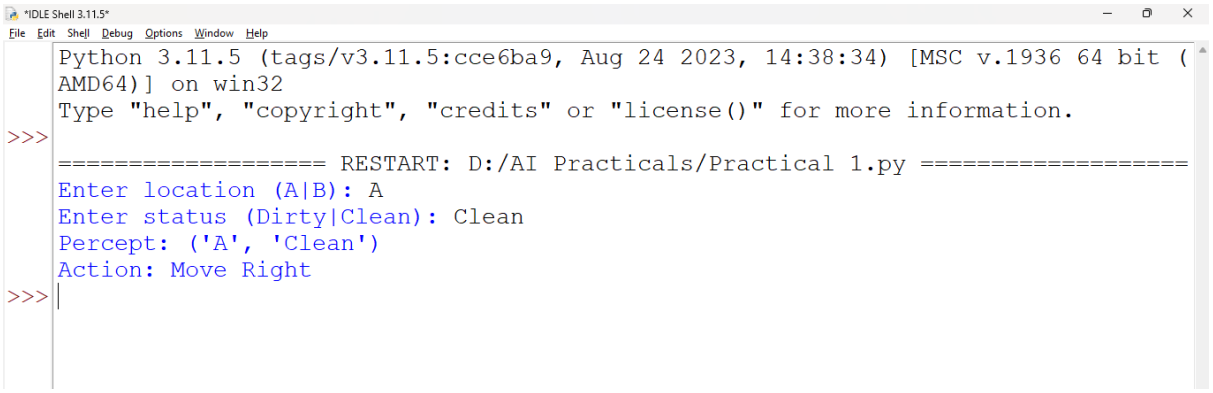
Percept History	Action
Dirty	Suck P: No B No S
Clean	Move Right

Code:-

```
table = {
    ("A", "Dirty"): "Suck",
    ("A", "Clean"): "Move Right",
    ("B", "Dirty"): "Suck",
    ("B", "Clean"): "Move Left"
}

location = input("Enter location (A|B): ")
status = input("Enter status (Dirty|Clean): ")
action = table.get((location, status), "Invalid Input")
print("Percept:", (location, status))
print("Action:", action)
```

Output:-



Practical No. 2

Aim:- Problem Solving by Searching.

a)Given and initial configuration of the 8-puzzle and a goal configuration, Write a python program to find the shortest sequence of moves to reach the goal state using breadth first search(BFS).

Code:-

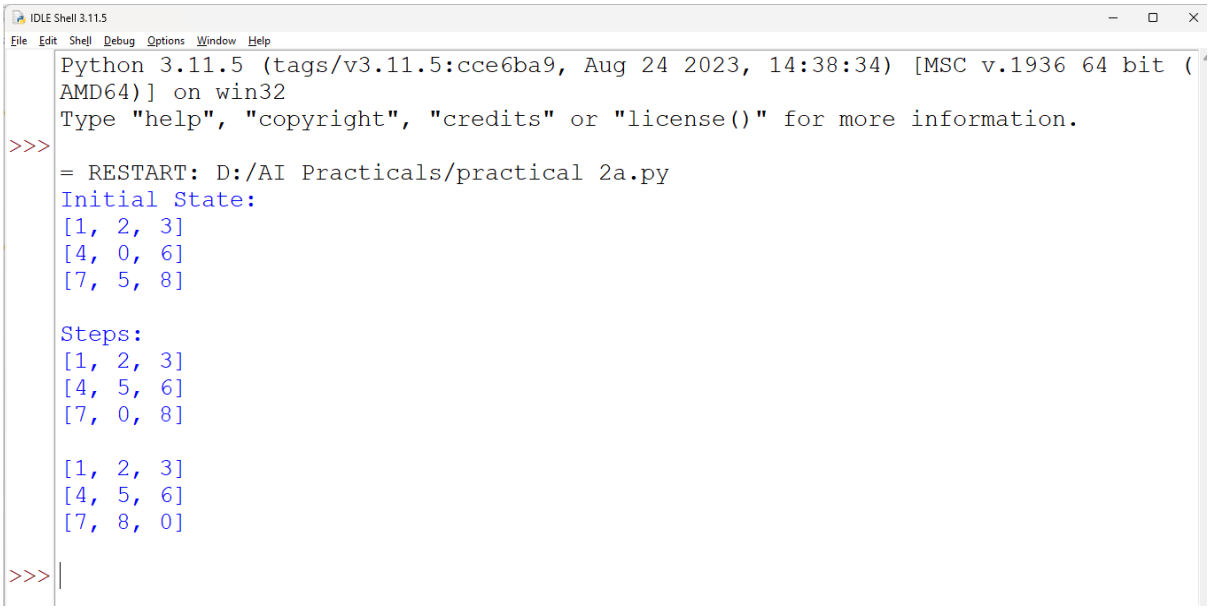
```
from collections import deque

def print_board(state):
    for i in range(0, 9, 3):
        print(state[i:i+3])
    print()

def bfs(start, goal):
    queue = deque([(start, [])])
    visited = []
    while queue:
        state, path = queue.popleft()
        if state == goal:
            return path
        if state not in visited:
            visited.append(state)
            pos = state.index(0)
            moves = []
            if pos > 2:
                moves.append(pos-3)
            if pos < 6:
                moves.append(pos+3)
            if pos % 3 != 0:
```

```
            moves.append(pos-1)
        if pos % 3 != 2:
            moves.append(pos+1)
        for move in moves:
            new_state = state[:]
            new_state[pos], new_state[move] = new_state[move], new_state[pos]
            queue.append((new_state, path+[new_state]))
    return None
start = [
    1, 2, 3,
    4, 0, 6,
    7, 5, 8
]
goal = [
    1, 2, 3,
    4, 5, 6,
    7, 8, 0
]
solution = bfs(start, goal)
print("Initial State:")
print_board(start)
if solution:
    print("Steps:")
    for step in solution:
        print_board(step)
else:
    print("No Solution Found")
```

Output:-


IDLE Shell 3.11.5
File Edit Shell Debug Options Window Help
Python 3.11.5 (tags/v3.11.5:cce6ba9, Aug 24 2023, 14:38:34) [MSC v.1936 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: D:/AI Practicals/practical 2a.py
Initial State:
[1, 2, 3]
[4, 0, 6]
[7, 5, 8]
Steps:
[1, 2, 3]
[4, 5, 6]
[7, 0, 8]
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
>>>

b)Given two water jugs of 4 litters and 3 litters capacity, Write a python program to obtain exactly 2 litters in 1 jug using Depth First Search(DFS). [Vary capacity of jugs]

Code:-

```
def dfs(x, y):
    if (x, y) in visited:
        return False
    visited.add((x, y))
    print("Jug1:", x, "Jug2:", y)
    if x == target or y == target:
        print("Goal Reached!")
        return True
    if dfs(cap1, y):
        return True
    if dfs(x, cap2):
        return True
    if dfs(0, y):
        return True
    if dfs(x, 0):
        return True
    transfer = min(x, cap2 - y)
    if dfs(x - transfer, y + transfer):
        return True
    transfer = min(y, cap1 - x)
    if dfs(x + transfer, y - transfer):
        return True
    return False
cap1 = 4
```

```
cap2 = 3

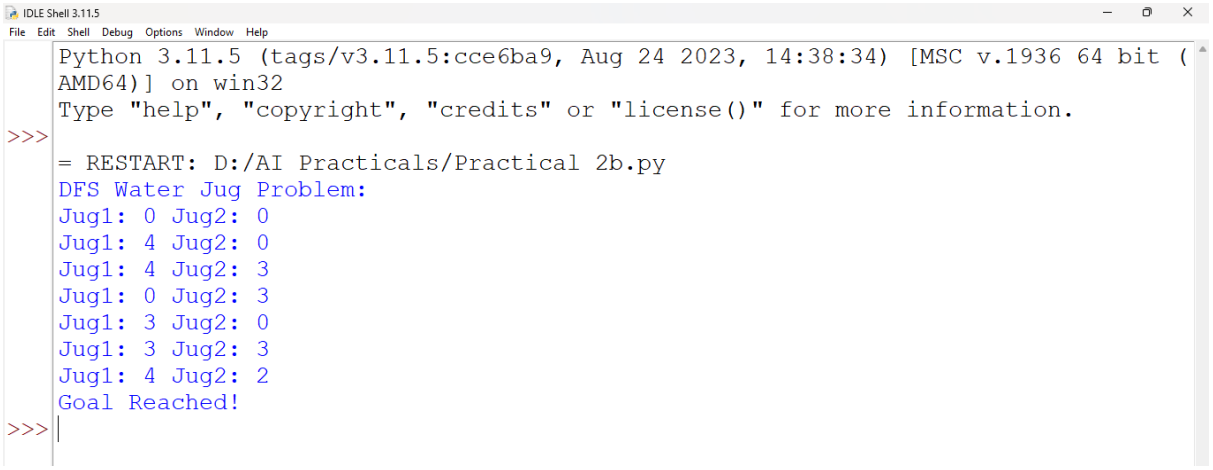
target = 2

visited = set()

print("DFS Water Jug Problem:")

dfs(0, 0)
```

Output:-



c) Given a Weighted graph representing cities and distances between them, Write a python program to find the least cost path from Arad to Bucharest using Uniform Cost Search (UCS). [Provide any other Weighted graph for applying UCS]

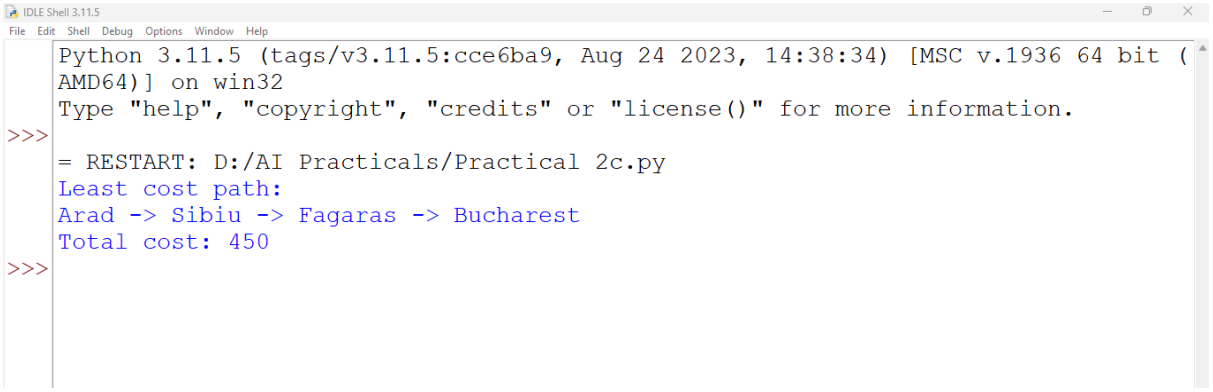
Code:-

```
import heapq

graph = {
    "Arad": [("Sibiu", 140), ("Zerind", 75), ("Timisoara", 118)],
    "Zerind": [("Arad", 75), ("Oradea", 71)],
    "Oradea": [("Zerind", 71), ("Sibiu", 151)],
    "Timisoara": [("Arad", 118), ("Lugoj", 111)],
    "Lugoj": [("Timisoara", 111), ("Mehadia", 70)],
    "Mehadia": [("Lugoj", 70), ("Drobeta", 75)],
    "Drobeta": [("Mehadia", 75), ("Craiova", 120)],
    "Craiova": [("Drobeta", 120), ("Pitesti", 138)],
    "Sibiu": [("Arad", 140), ("Fagaras", 99)],
    "Fagaras": [("Sibiu", 99), ("Bucharest", 211)],
    "Pitesti": [("Craiova", 138), ("Bucharest", 101)],
    "Bucharest": []
}
def ucs(start, goal):
    queue = [(0, start, [start])]
    visited = set()
    while queue:
        cost, node, path = heapq.heappop(queue)
        if node == goal:
            return cost, path
        if node in visited:
            continue
```

```
        visited.add(node)
        for neighbor, weight in graph[node]:
            new_cost = cost + weight
            heapq.heappush(
                queue,
                (new_cost, neighbor, path + [neighbor])
            )
    return None
result = ucs("Arad", "Bucharest")
print("Least cost path:")
print(" -> ".join(result[1]))
print("Total cost:", result[0])
```

Output:-



Practical No. 3

Aim:- Problem Solving by Searching (Informed Search).

a) Given an initial configuration of the 8-puzzle and goal configuration , Write a python program to find the solution using Greedy breadth Search with the Manhattan Distance Heuristic.

Code:-

```
from heapq import heappush, heappop

GOAL = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 0]
]

def manhattan(state):
    distance = 0
    for i in range(3):
        for j in range(3):
            value = state[i][j]
            if value != 0:
                goal_x = (value - 1) // 3
                goal_y = (value - 1) % 3
                distance += abs(i - goal_x) + abs(j - goal_y)
    return distance

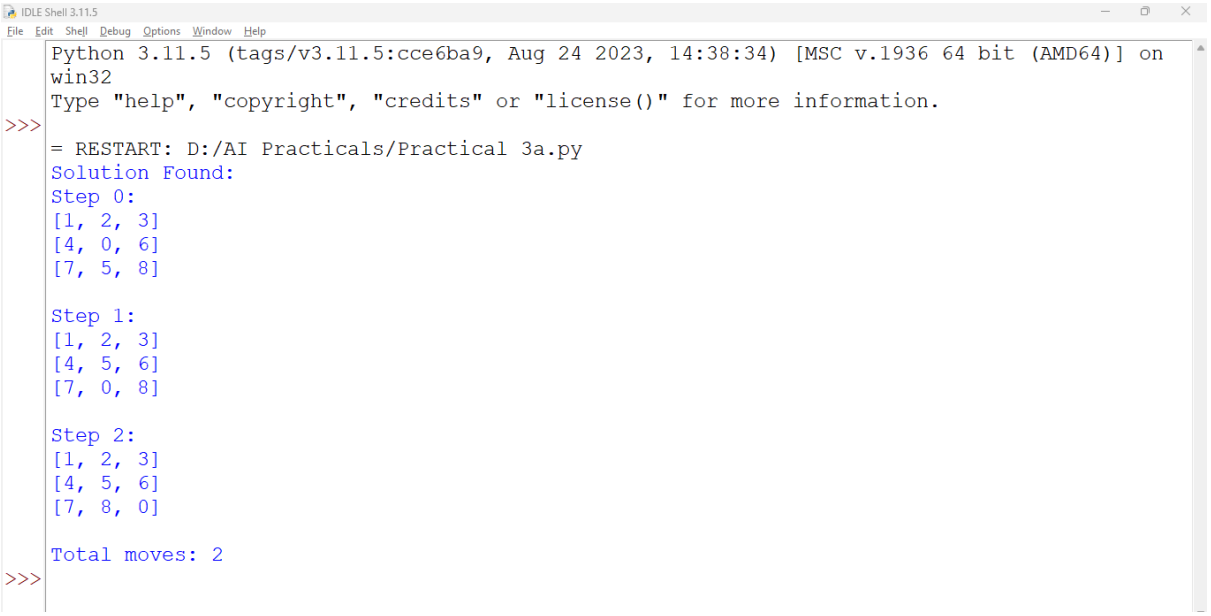
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j
```

```
    return None
def get_neighbors(state):
    neighbors = []
    x, y = find_blank(state)
    moves = [
        (-1, 0),
        (1, 0),
        (0, -1),
        (0, 1)
    ]
    for dx, dy in moves:
        nx = x + dx
        ny = y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [row[:] for row in state]
            new_state[x][y], new_state[nx][ny] = \
                new_state[nx][ny], new_state[x][y]
            neighbors.append(new_state)
    return neighbors
def state_to_tuple(state):
    return tuple(tuple(row) for row in state)
def greedy_best_first_search(start):
    pq = []
    heappush(
        pq,
        (manhattan(start), start, [start])
```

```
    )
    visited = set()
    while pq:
        h, current, path = heappop(pq)
        if current == GOAL:
            return path
        state_key = state_to_tuple(current)
        if state_key in visited:
            continue
        visited.add(state_key)
        for neighbor in get_neighbors(current):
            neighbor_key = state_to_tuple(neighbor)
            if neighbor_key not in visited:
                heappush(
                    pq,
                    (
                        manhattan(neighbor),
                        neighbor,
                        path + [neighbor]
                    )
                )
    return None
def print_state(state):
    for row in state:
        print(row)
    print()
start_state = [
```

```
    [1, 2, 3],
    [4, 0, 6],
    [7, 5, 8]
]
solution = greedy_best_first_search(start_state)
if solution:
    print("Solution Found:")
    for step, state in enumerate(solution):
        print(f"Step {step}:")
        print_state(state)
    print("Total moves:", len(solution) - 1)
else:
    print("No Solution Found")
```

Output:-



b) Given an initial configuration of the 8-puzzle and a goal configuration, write a Python program to find the shortest path using A* search with the Manhattan Distance Heuristic (MDH).

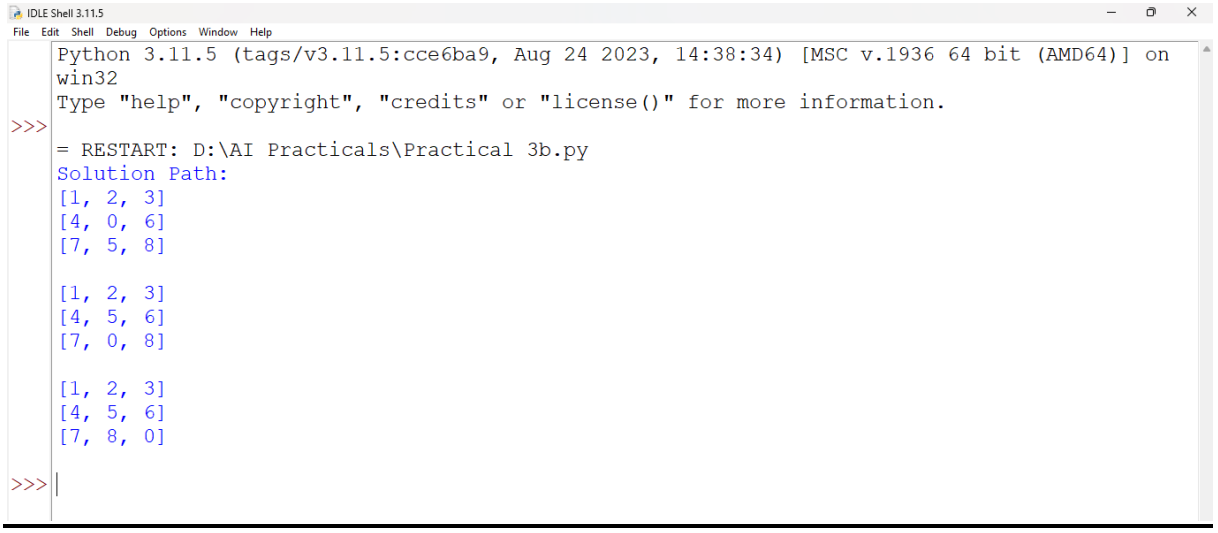
Code:-

```
from heapq import heappush, heappop
goal = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 0]
]
def heuristic(state):
    h = 0
    for i in range(3):
        for j in range(3):
            val = state[i][j]
            if val != 0:
                x = (val - 1) // 3
                y = (val - 1) % 3
                h += abs(i - x) + abs(j - y)
    return h
def blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j
def neighbors(state):
    x, y = blank(state)
    moves = [
```

```
        (-1, 0),
        (1, 0),
        (0, -1),
        (0, 1)
    ]
    result = []
    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [row[:] for row in state]
            new_state[x][y], new_state[nx][ny] = \
                new_state[nx][ny], new_state[x][y]
            result.append(new_state)
    return result
def astar(start):
    pq = []
    heappush(
        pq,
        (heuristic(start), 0, start, [start])
    )
    visited = []
    while pq:
        f, g, state, path = heappop(pq)
        if state == goal:
            return path
        if state in visited:
            continue
```

```
        visited.append(state)
        for child in neighbors(state):
            heappush(
                pq,
                (
                    g + 1 + heuristic(child),
                    g + 1,
                    child,
                    path + [child]
                )
            )
    return None
start = [
    [1, 2, 3],
    [4, 0, 6],
    [7, 5, 8]
]
solution = astar(start)
print("Solution Path:")
for step in solution:
    for row in step:
        print(row)
    print()
```

Output:-


IDLE Shell 3.11.5
File Edit Shell Debug Options Window Help
Python 3.11.5 (tags/v3.11.5:cce6ba9, Aug 24 2023, 14:38:34) [MSC v.1936 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: D:\AI Practicals\Practical 3b.py
Solution Path:
[1, 2, 3]
[4, 0, 6]
[7, 5, 8]
[1, 2, 3]
[4, 5, 6]
[7, 0, 8]
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
>>>

c) Given a weighted graph representing cities and distance between them. Write a python program find the shortest path from Arad to Bucharest using A* search with a Heuristic function (Straight line distance to destination).

Code:-

```
from heapq import heappush, heappop

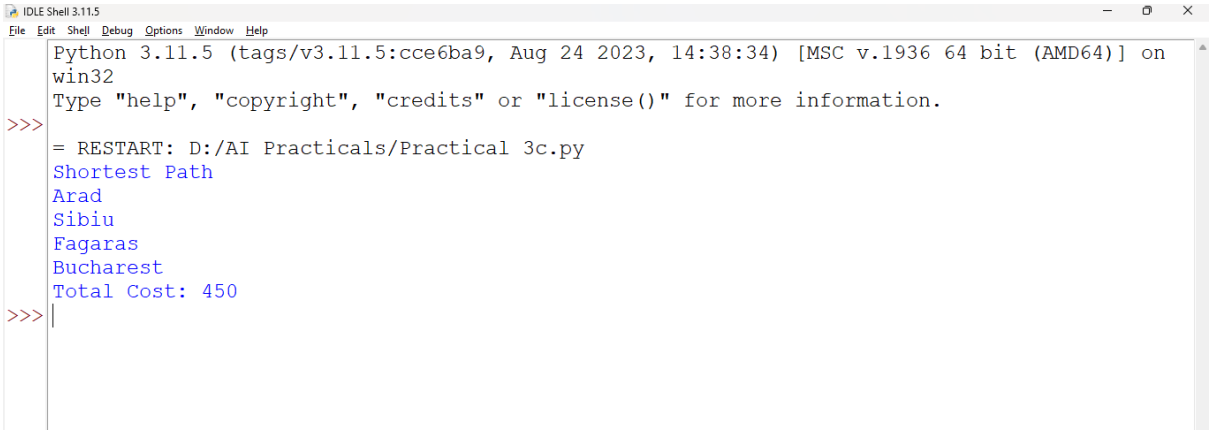
graph = {
    "Arad": [("Sibiu", 140), ("Zerind", 75), ("Timisoara", 118)],
    "Zerind": [("Arad", 75), ("Oradea", 71)],
    "Oradea": [("Zerind", 71), ("Sibiu", 151)],
    "Timisoara": [("Arad", 118), ("Lugoj", 111)],
    "Lugoj": [("Timisoara", 111), ("Mehadia", 70)],
    "Mehadia": [("Lugoj", 70), ("Drobeta", 75)],
    "Drobeta": [("Mehadia", 75), ("Craiova", 120)],
    "Craiova": [("Drobeta", 120), ("Pitesti", 138)],
    "Sibiu": [("Arad", 140), ("Fagaras", 99)],
    "Fagaras": [("Sibiu", 99), ("Bucharest", 211)],
    "Pitesti":[("Raiova",128),("Bucharest",101)],
    "Bucharest": []
}
heuristic = {
    'Arad': 366,
    'Zerind': 374,
    'Oradea': 380,
    'Timisoara': 329,
    'Lugoj': 244,
    'Mehadia': 241,
    'Drobeta': 242,
    'Craiova': 160,
```

```
    'Sibiu': 253,
    'Rimina':193,
    'Fagaras': 176,
    'Pitesti': 100,
    'Bucharest': 0
}
def a_star(start, goal):
    pq = []
    heappush(pq, (heuristic[start], 0, start, [start]))
    visited = set()
    while pq:
        f, g, city, path = heappop(pq)
        if city == goal:
            return path, g
        if city in visited:
            continue
        visited.add(city)
        for neighbor, cost in graph[city]:
            if neighbor not in visited:
                new_g = g + cost
                new_f = new_g + heuristic[neighbor]
                heappush(pq, (new_f, new_g, neighbor,
                              path + [neighbor]))
    return None, float("inf")
path, cost = a_star("Arad", "Bucharest")
print("Shortest Path")
for city in path:
```

```
    print(city)

print("Total Cost:", cost)
```

Output:-



Practical No. 4

Aim:- Bayes Rule Application.

a) Write a python program to compute the probability that the person actually have the disease the test result is positive.

Code:-

```
P_D=0.01

P_notD=0.99

P_T_given_D=0.99

P_T_given_notD=0.05

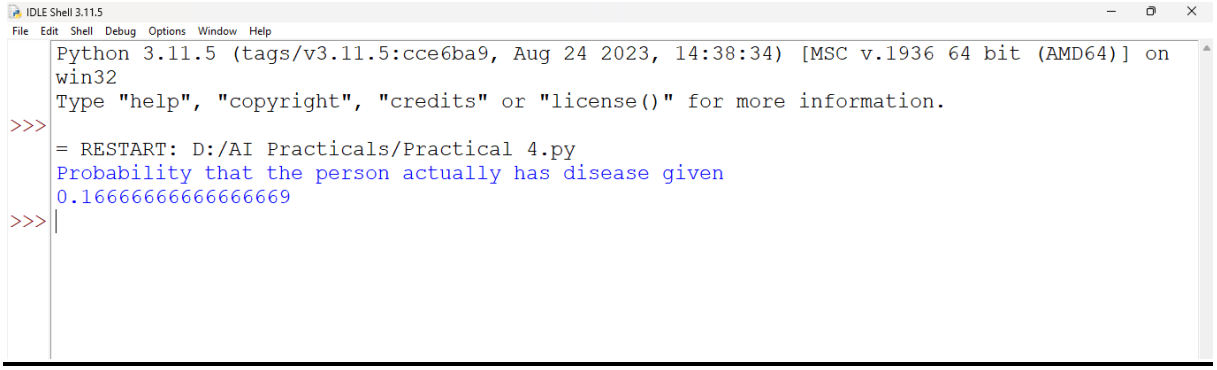
P_T=(P_T_given_D*P_D)+(P_T_given_notD*P_notD)

P_D_given_T=(P_T_given_D*P_D)/P_T

print("Probability that the person actually has disease given")

print(P_D_given_T)
```

Output:-



Practical No. 5

Aim:- Naïve Bayes Classification.

a) Using a small Datasets(Example: Whether or Spam Dataset), Implement a Naive Bayes Classification using scikit-learn.

Code:-

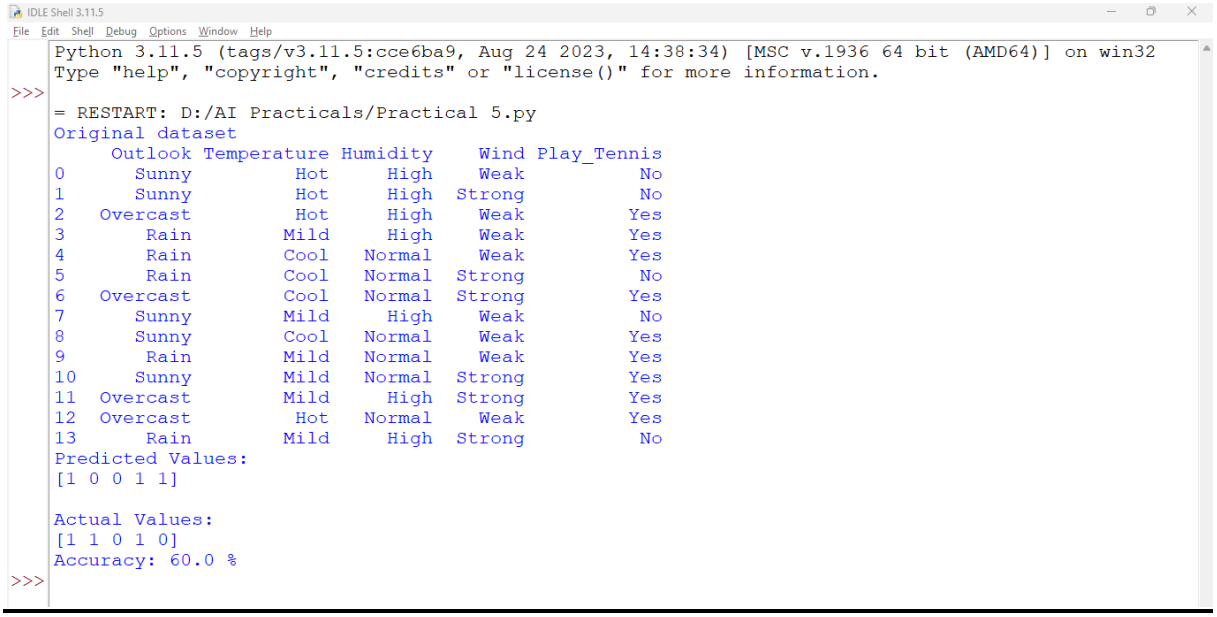
```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
df=pd.read_csv("PlayTennis.csv")
print("Original dataset")
print(df)
encoder=LabelEncoder()
for column in df.columns:
    df[column]=encoder.fit_transform(df[column])
x = df[['Outlook', 'Temperature', 'Humidity', 'Wind']]
y = df['Play_Tennis']
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.3, random_state=42
)
model = GaussianNB()
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
print("Predicted Values:")
print(y_pred)
print("\nActual Values:")
```

```
print(y_test.values)

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy * 100, "%")
```

Output:-



```
IDLE Shell 3.11.5
File Edit Shell Debug Options Window Help
Python 3.11.5 (tags/v3.11.5:cce6ba9, Aug 24 2023, 14:38:34) [MSC v.1936 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: D:/AI Practicals/Practical 5.py
Original dataset
     Outlook Temperature Humidity    Wind Play_Tennis
0      Sunny         Hot     High    Weak          No
1      Sunny         Hot     High  Strong          No
2   Overcast         Hot     High    Weak         Yes
3       Rain        Mild     High    Weak         Yes
4       Rain        Cool   Normal    Weak         Yes
5       Rain        Cool   Normal  Strong          No
6   Overcast        Cool   Normal  Strong         Yes
7      Sunny        Mild     High    Weak          No
8      Sunny        Cool   Normal    Weak         Yes
9       Rain        Mild   Normal    Weak         Yes
10     Sunny        Mild   Normal  Strong         Yes
11  Overcast        Mild     High  Strong         Yes
12  Overcast         Hot   Normal    Weak         Yes
13      Rain        Mild     High  Strong          No
Predicted Values:
[1 0 0 1 1]

Actual Values:
[1 1 0 1 0]
Accuracy: 60.0 %
>>>
```

Practical No. 6

Aim:- Probability Reasoning (Baysian Network & Inferences).

a) Write a program to compute posterior probability using Bayes. (Burglary/Alarm=True)

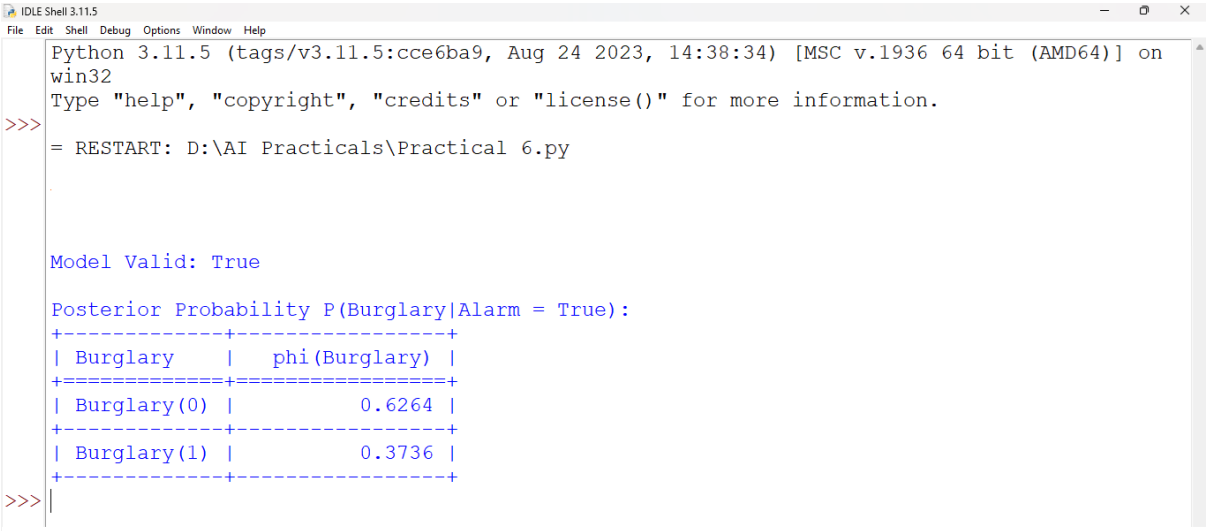
Code:-

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination


model = DiscreteBayesianNetwork([
    ('Burglary', 'Alarm'),
    ('Earthquake', 'Alarm')
])
cpd_burglary = TabularCPD(
    variable='Burglary',
    variable_card=2,
    values=[[0.999], [0.001]]
)
cpd_earthquake = TabularCPD(
    variable='Earthquake',
    variable_card=2,
    values=[[0.998], [0.002]]
)
cpd_alarm = TabularCPD(
    variable='Alarm',
    variable_card=2,
    values=[[0.999, 0.71, 0.06, 0.05],
            [0.001, 0.29, 0.94, 0.95]],
```

```
    evidence=['Burglary', 'Earthquake'],
    evidence_card=[2, 2]
)
model.add_cpds(cpd_burglary, cpd_earthquake, cpd_alarm)
print("Model Valid:", model.check_model())
inference = VariableElimination(model)
result = inference.query(
    variables=['Burglary'],
    evidence={'Alarm': 1}
)
print("\nPosterior Probability P(Burglary|Alarm = True):")
print(result)
```

Output:-



Practical No. 7

Aim:- Machine Learning.

a) Using any small dataset (e.g., Iris or a CSV dataset), write a python program to demonstrate the basic machine learning workflow.

1. Load dataset using Pandas
2. Perform basic preprocessing (handling missing values or scaling).
3. Split dataset into training and testing sets.
4. Train a simple classifier.
5. Display training and testing accuracy.

Code:-

```
import pandas as pd

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score


iris = load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)

df['target'] = iris.target


print("First 5 rows of Dataset:")

print(df.head())


X = df.drop('target', axis=1)

y = df['target']


scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42)

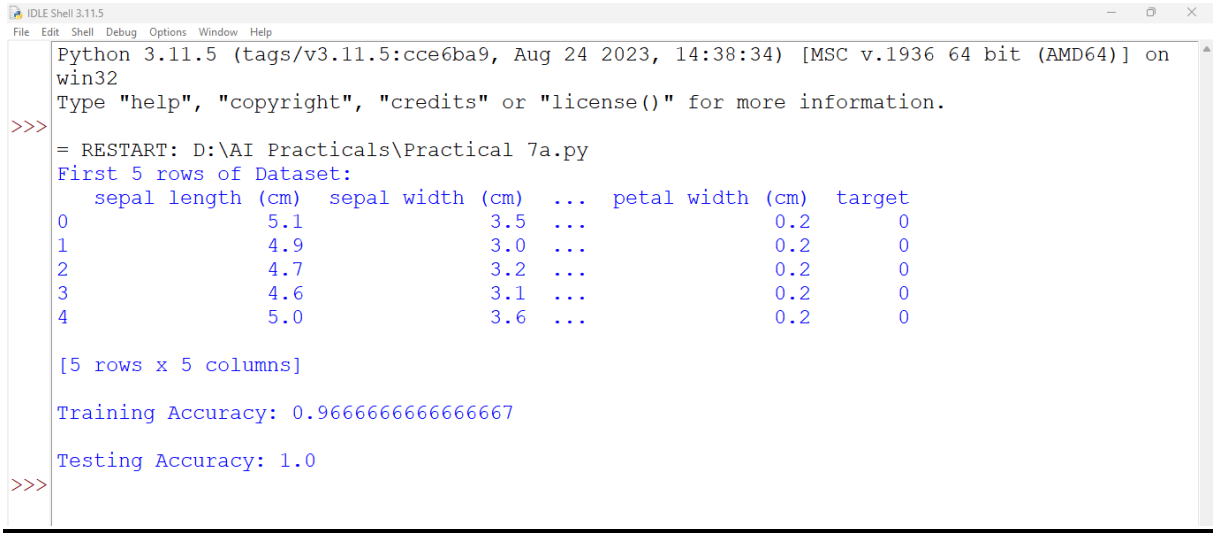
model = LogisticRegression()
model.fit(X_train, y_train)

y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

train_acc = accuracy_score(y_train, y_train_pred)
test_acc = accuracy_score(y_test, y_test_pred)

print("\nTraining Accuracy:", train_acc)
print("\nTesting Accuracy:", test_acc)
```

Output:-



b) Using a suitable regression dataset, implement Linear Regression using scikit-learn.

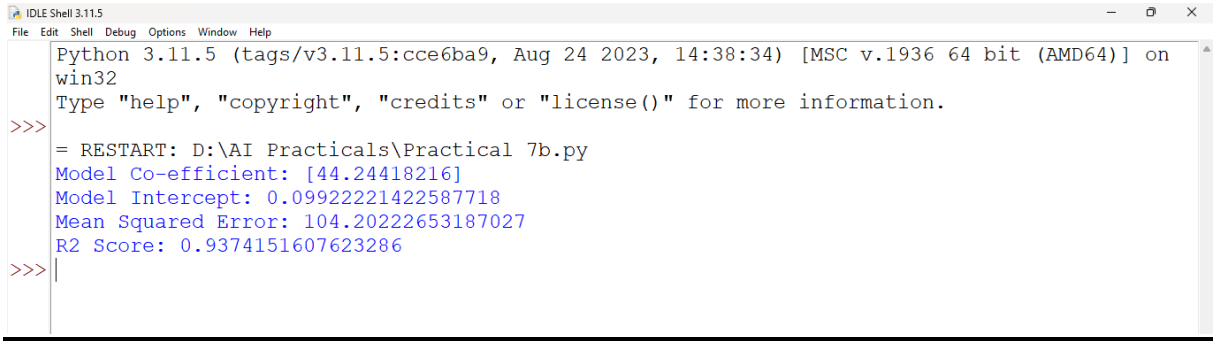
Code:-

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.datasets import make_regression
from sklearn.metrics import mean_squared_error,r2_score

x,y=make_regression(
    n_samples=100,
    n_features=1,
    noise=10,
    random_state=42)

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42)
model=LinearRegression()
model.fit(x_train,y_train)
y_pred=model.predict(x_test)
mse=mean_squared_error(y_test,y_pred)
r2=r2_score(y_test,y_pred)
print("Model Co-efficient:",model.coef_)
print("Model Intercept:",model.intercept_)
print("Mean Squared Error:",mse)
print("R2 Score:",r2)
```

Output:-


IDLE Shell 3.11.5
File Edit Shell Debug Options Window Help
Python 3.11.5 (tags/v3.11.5:cce6ba9, Aug 24 2023, 14:38:34) [MSC v.1936 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: D:\AI Practicals\Practical 7b.py
Model Co-efficient: [44.24418216]
Model Intercept: 0.09922221422587718
Mean Squared Error: 104.20222653187027
R2 Score: 0.9374151607623286
>>>

c) Using the MNIST dataset (digit vs not digit), implement a binary classifier.

Code:-

```
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.linear_model import SGDClassifier

mnist = fetch_openml("mnist_784", version=1,
                     as_frame=False)

x = mnist.data
y = mnist.target.astype(int)
y = (y == 5)

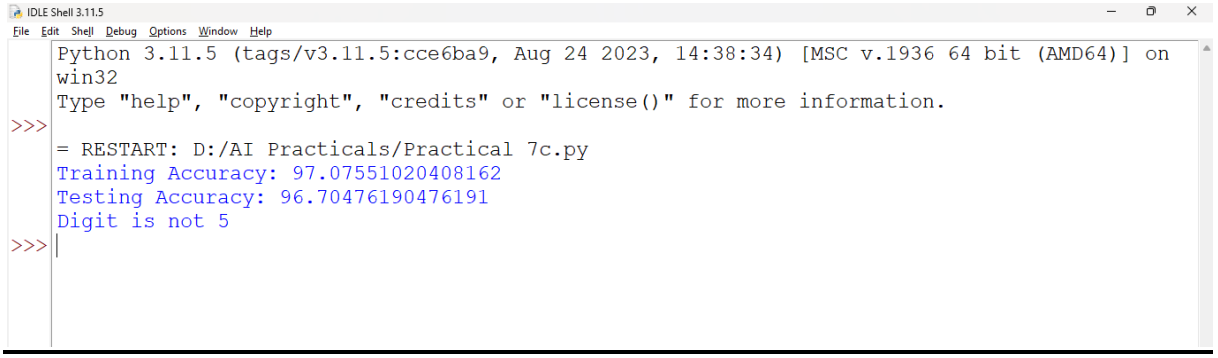
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.3, random_state=1)

model = SGDClassifier(random_state=1)
model.fit(x_train, y_train)

print("Training Accuracy:", model.score(x_train, y_train) * 100)
print("Testing Accuracy:", model.score(x_test, y_test) * 100)
prediction = model.predict([x_test[0]])

if prediction[0]:
    print("Digit is 5")
else:
    print("Digit is not 5")
```

Output:-

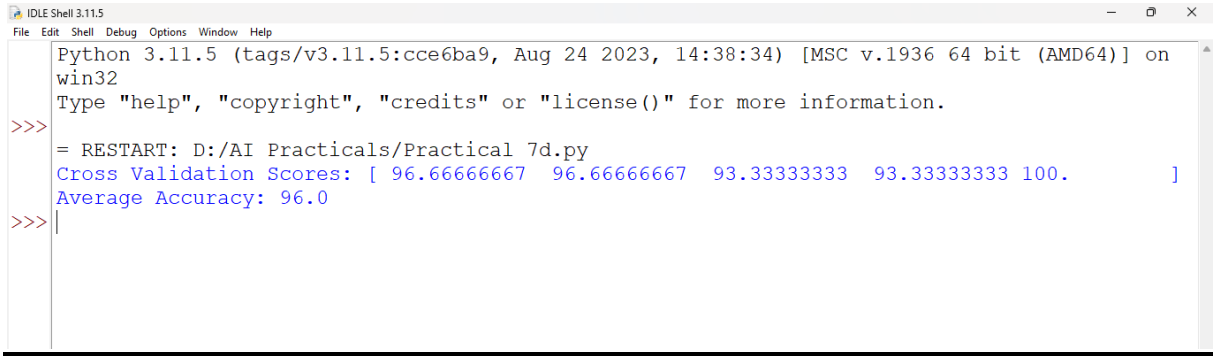

IDLE Shell 3.11.5
File Edit Shell Debug Options Window Help
Python 3.11.5 (tags/v3.11.5:cce6ba9, Aug 24 2023, 14:38:34) [MSC v.1936 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: D:/AI Practicals/Practical 7c.py
Training Accuracy: 97.07551020408162
Testing Accuracy: 96.70476190476191
Digit is not 5
>>>

d) Implement k-fold cross-validation on a classification model.

Code:-

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split,KFold,cross_val_score
from sklearn.linear_model import LogisticRegression
data = load_iris()
x = data.data
y = data.target
model = LogisticRegression(max_iter=1000)
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=1)
model = LogisticRegression(max_iter=1000)
KFold = KFold(n_splits=5, shuffle=True, random_state=1)
scores = cross_val_score(model, x, y, cv=KFold)
print("Cross Validation Scores:", scores * 100)
print("Average Accuracy:", scores.mean() * 100)
```

Output:-

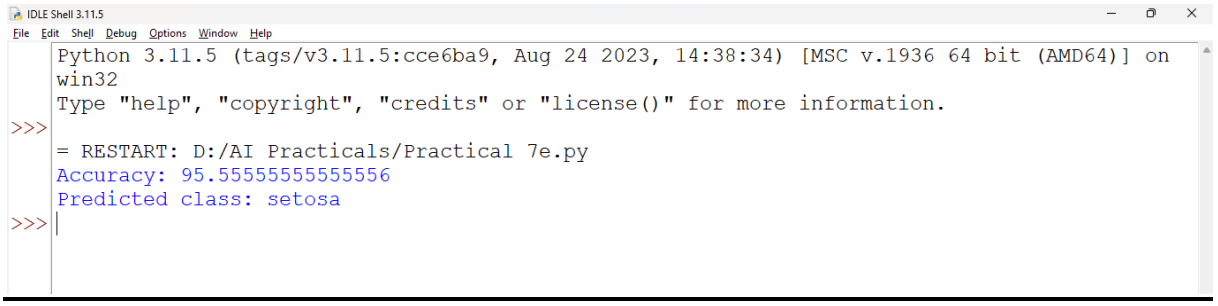


e) Implement a Decision Tree classifier using the Iris or any relevant dataset.

Code:-

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.tree import DecisionTreeClassifier
iris = load_iris()
x = iris.data
y = iris.target
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size = 0.3, random_state=1)
model = DecisionTreeClassifier(random_state=1)
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
print("Accuracy:", accuracy_score(y_test, y_pred)*100)
prediction = model.predict([x_test[0]])
print("Predicted class:", iris.target_names[prediction[0]])
```

Output:-



f)Implement Decision Tree regression on a regression dataset.

Code:-

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

from sklearn.tree import DecisionTreeRegressor

data = fetch_california_housing()

x = data.data
y = data.target

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.3, random_state=1)

model = DecisionTreeRegressor(random_state=1)

model.fit(x_train, y_train)

y_pred = model.predict(x_test)

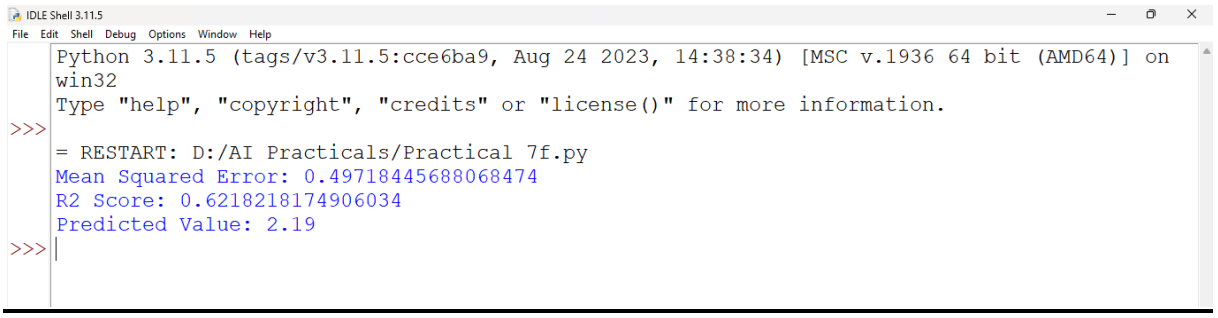
print("Mean Squared Error:", mean_squared_error
      (y_test, y_pred))

print("R2 Score:", r2_score(y_test, y_pred))
```

```
prediction = model.predict([x_test[0]])


print("Predicted Value:", prediction[0])
```

Output:-



g) Implement k-Nearest Neighbors (kNN) classifier.

Code:-

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.neighbors import KNeighborsClassifier

iris = load_iris()

x = iris.data
y = iris.target

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42)

knn = KNeighborsClassifier(n_neighbors=3)

knn.fit(x_train, y_train)

train_pred = knn.predict(x_train)

test_pred = knn.predict(x_test)

train_accuracy = accuracy_score(y_train, train_pred)

test_accuracy = accuracy_score(y_test, test_pred)
```

```
print("Training Accuracy:", train_accuracy)

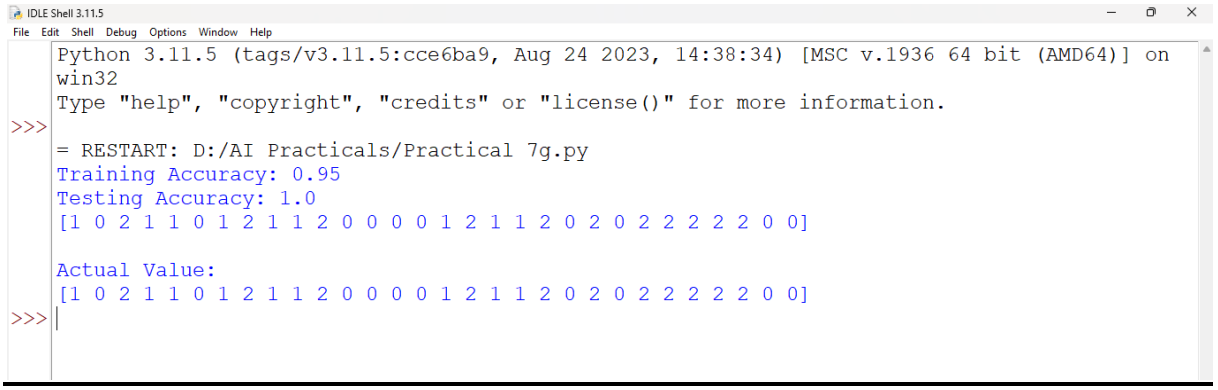
print("Testing Accuracy:", test_accuracy)

print(test_pred)

print("\nActual Value:")

print(y_test)
```

Output:-

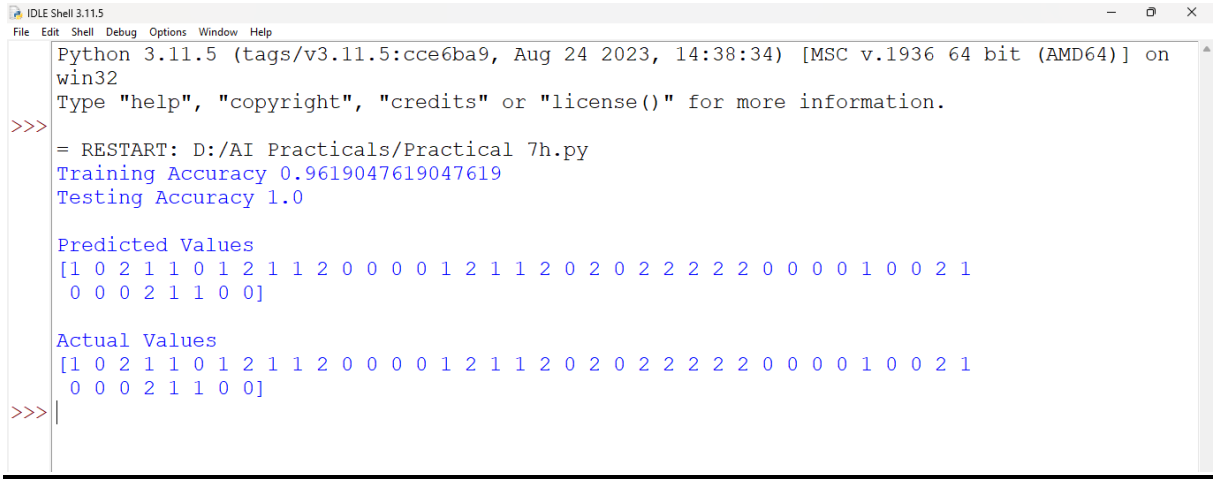


h) Implement multiclass classification using Logistic Regression or Decision Tree.

Code:-

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
iris = load_iris()
x = iris.data
y = iris.target
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size = 0.3, random_state = 42)
model = LogisticRegression(max_iter = 200)
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
train_pred = model.predict(x_train)
test_pred = model.predict(x_test)
print("Training Accuracy", accuracy_score(y_train,
                                          train_pred))
print("Testing Accuracy", accuracy_score(y_test,
                                         test_pred))
print("\nPredicted Values")
print(test_pred)
print("\nActual Values")
print(y_test)
```

Output:-


IDLE Shell 3.11.5
File Edit Shell Debug Options Window Help
Python 3.11.5 (tags/v3.11.5:cce6ba9, Aug 24 2023, 14:38:34) [MSC v.1936 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: D:/AI Practicals/Practical 7h.py
Training Accuracy 0.9619047619047619
Testing Accuracy 1.0
Predicted Values
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
0 0 0 2 1 1 0 0]
Actual Values
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
0 0 0 2 1 1 0 0]
>>>

Practical No. 8

Aim:- Ethics and Bias Analysis.

a) Select any publicly available dataset (for example, Iris, Adult Income, or a classification dataset of your choice) and analyze it for possible bias and class imbalance.

Code:-

```
import pandas as pd

from sklearn.datasets import fetch_openml

import matplotlib.pyplot as plt

data = fetch_openml('adult', version=2, as_frame=True)

df = data.frame

print("Dataset Shape:", df.shape)

print("\nClass Distribution")

print(df['class'].value_counts())

df['class'].value_counts().plot(kind='bar')

plt.title("Income Class Distribution")

plt.xlabel("Income class")

plt.ylabel("Count")

plt.show()

print("\nIncome distribution by gender:")

print(pd.crosstab(df['sex'], df['class']))

print("\nIncome distribution by race:")

print(pd.crosstab(df['race'], df['class']))
```

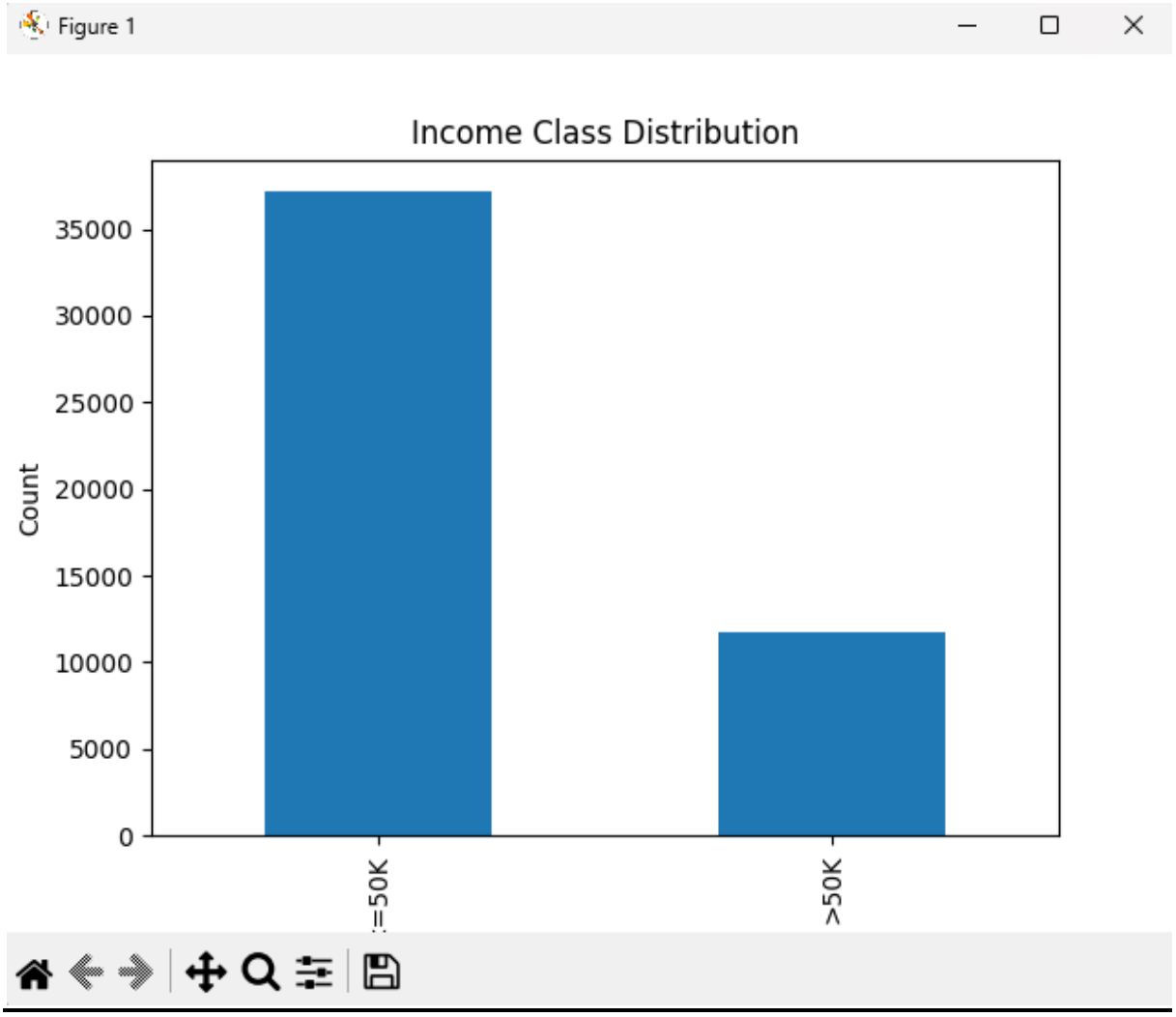
Output:-

```
IDLE Shell 3.11.5
File  Edit  Shell  Debug  Options  Window  Help
Python 3.11.5 (tags/v3.11.5:cce6ba9, Aug 24 2023, 14:38:34) [MSC v.1936 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: D:/AI Practicals/Practical 8.py
Dataset Shape: (48842, 15)

Class Distribution
class
<=50K    37155
>50K     11687
Name: count, dtype: int64

Income distribution by gender:
class   <=50K  >50K
sex
Female  14423  1769
Male    22732  9918

Income distribution by race:
class               <=50K   >50K
race
Amer-Indian-Eskimo    415     55
Asian-Pac-Islander   1110    409
Black                4119    566
Other                 356     50
White               31155  10607
>>>
```



MODULE-2

Practical No. 1

Aim:- Jira setup & Project Creation.

a) Create a Jira Cloud account and explore the dashboard

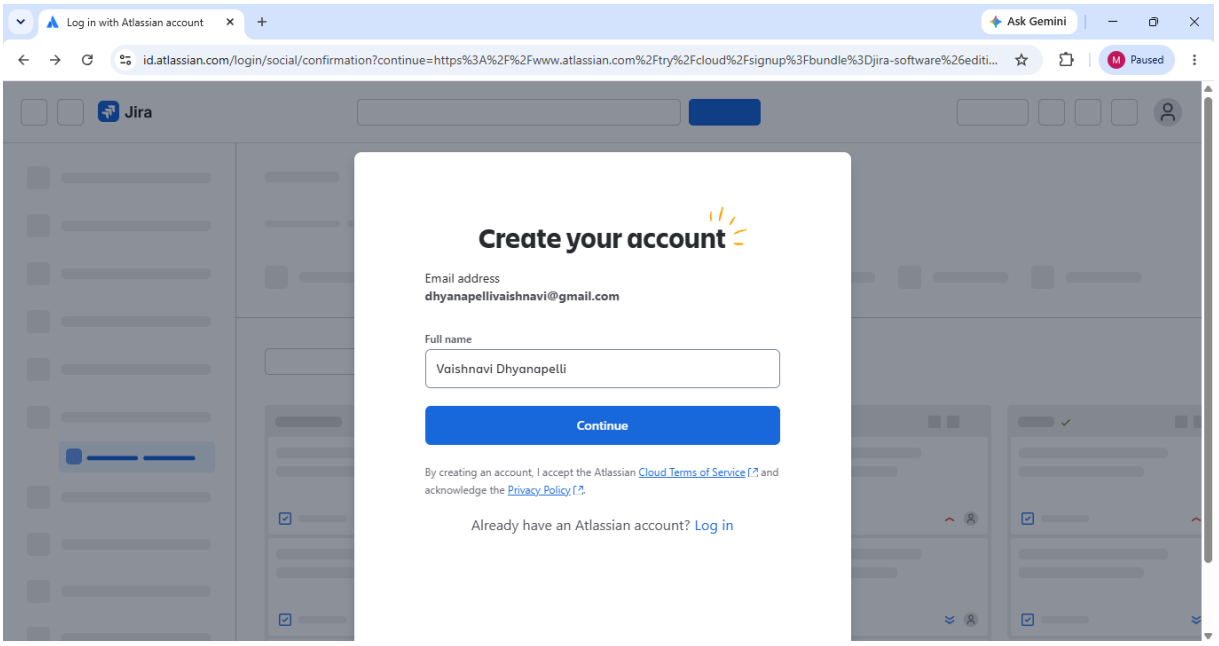
Steps:-

1. Open Google Chrome and Search for Jira.

2. Click on the Jira website and select Log in with Google.

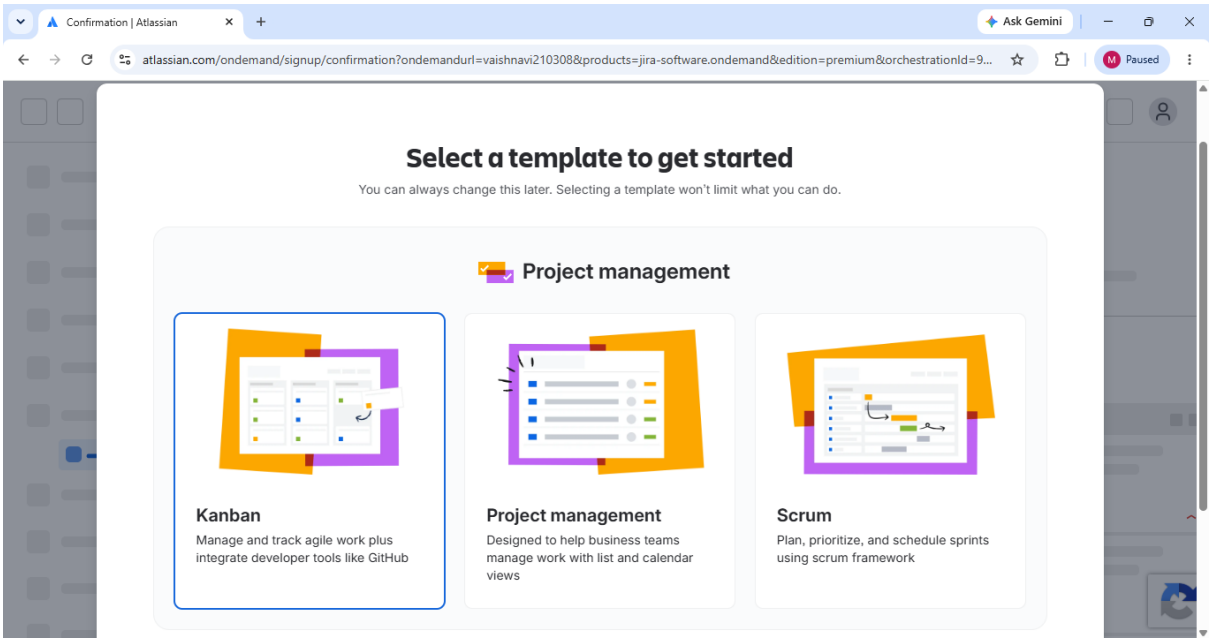
3. Log in using your Gmail ID.

4. The Create Your account screen will appear. Click on Continue.



5. Select Project Management from the list and Click on Continue.

6. Select Kanban from the list and click on Continue.

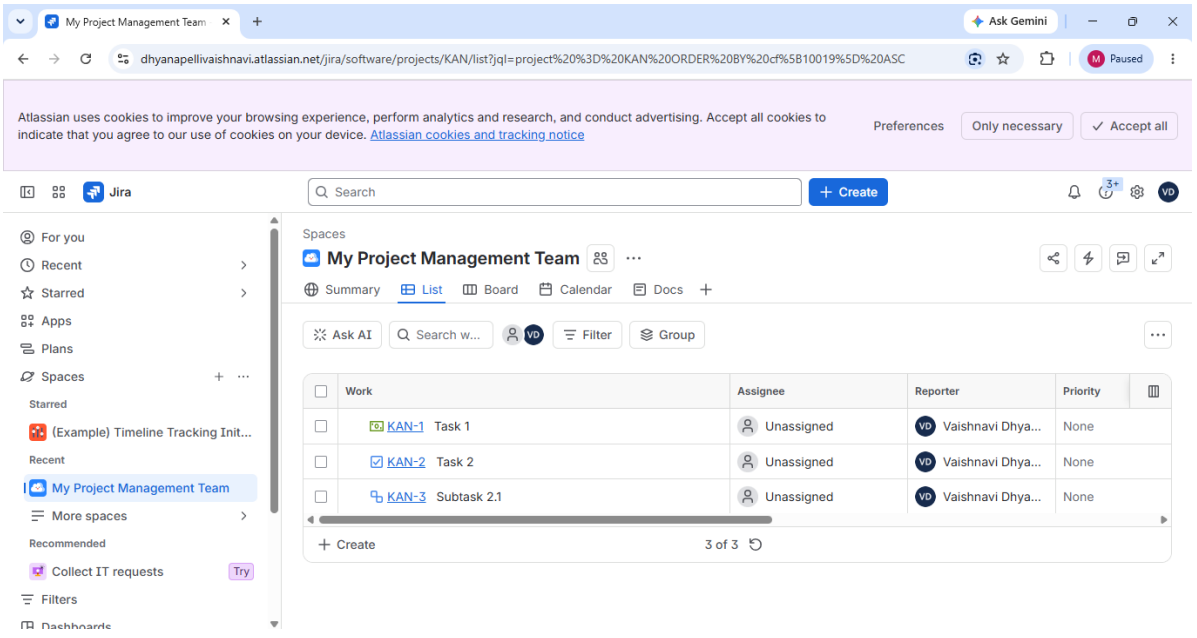


7. Provide the required project details and click on Get Started.

8. Click on Next and then click on Next again.

9. Click on Go to Jira.

10. The Jira Dashboard will appear.



b) Create a Company-Managed Scrum or Kanban project.

Steps:-

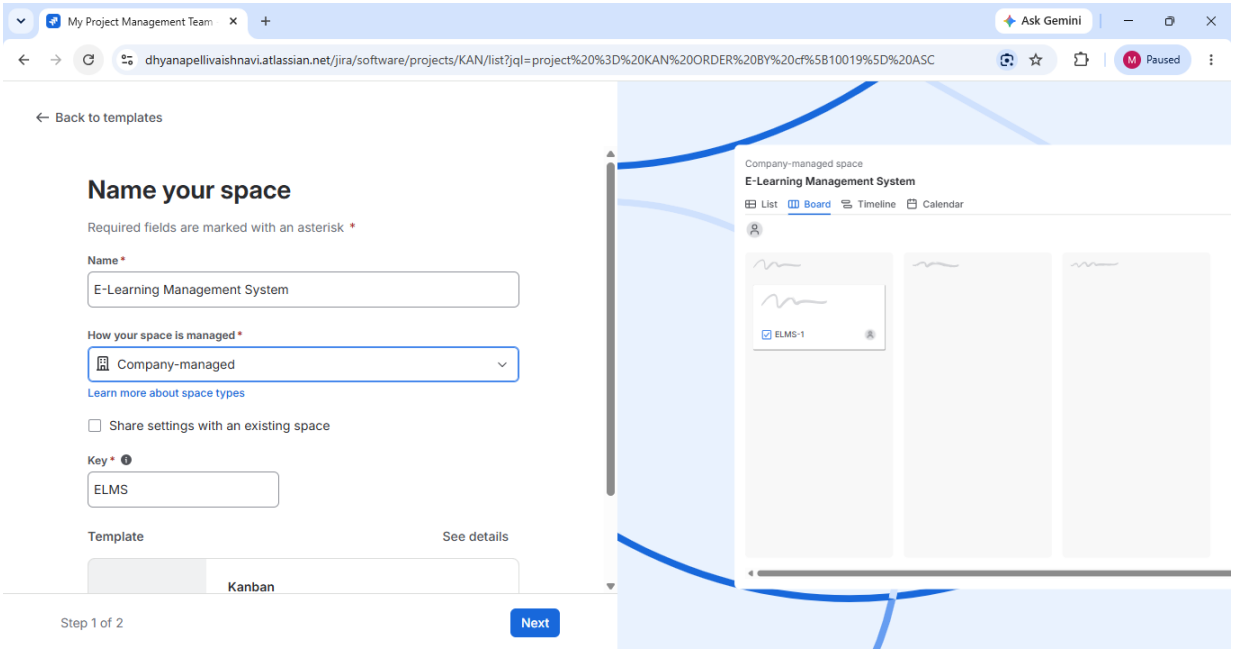
1. Click on the Project option and select Project Settings.

2. Go to Space Settings**.**

3. Click on Create Space.

4. Select Kanban and click on Use Template, then provide the required details.

5. Select Company-Managed in the space management option.



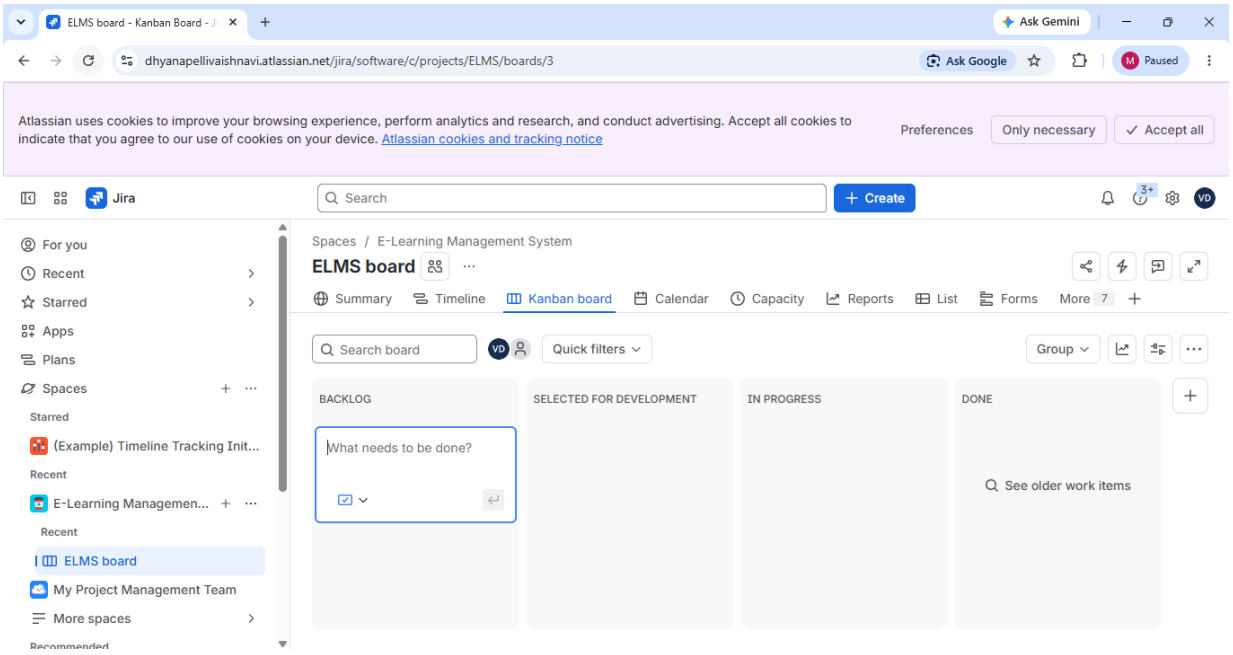
6. Click on Next.

7. Add the email addresses of your team members.

8. Click on I’ll do it later if you want to skip adding team members.

9. Click on Next.

10. The Project Dashboard will appear.


ELMS board - Kanban Board - J
Ask Gemini
dhyanapellivaishnavi.atlassian.net/jira/software/c/projects/ELMS/boards/3
Ask Google
Paused
Atlassian uses cookies to improve your browsing experience, perform analytics and research, and conduct advertising. Accept all cookies to indicate that you agree to our use of cookies on your device. Atlassian cookies and tracking notice
Preferences
Only necessary
Accept all
Jira
Search
Create
For you
Recent
Starred
Apps
Plans
Spaces
Starred
(Example) Timeline Tracking Init...
Recent
E-Learning Managemen...
Recent
ELMS board
My Project Management Team
More spaces
Recommended
Spaces / E-Learning Management System
ELMS board
Summary
Timeline
Kanban board
Calendar
Capacity
Reports
List
Forms
More 7
Search board
Quick filters
Group
BACKLOG
What needs to be done?
SELECTED FOR DEVELOPMENT
IN PROGRESS
DONE
See older work items

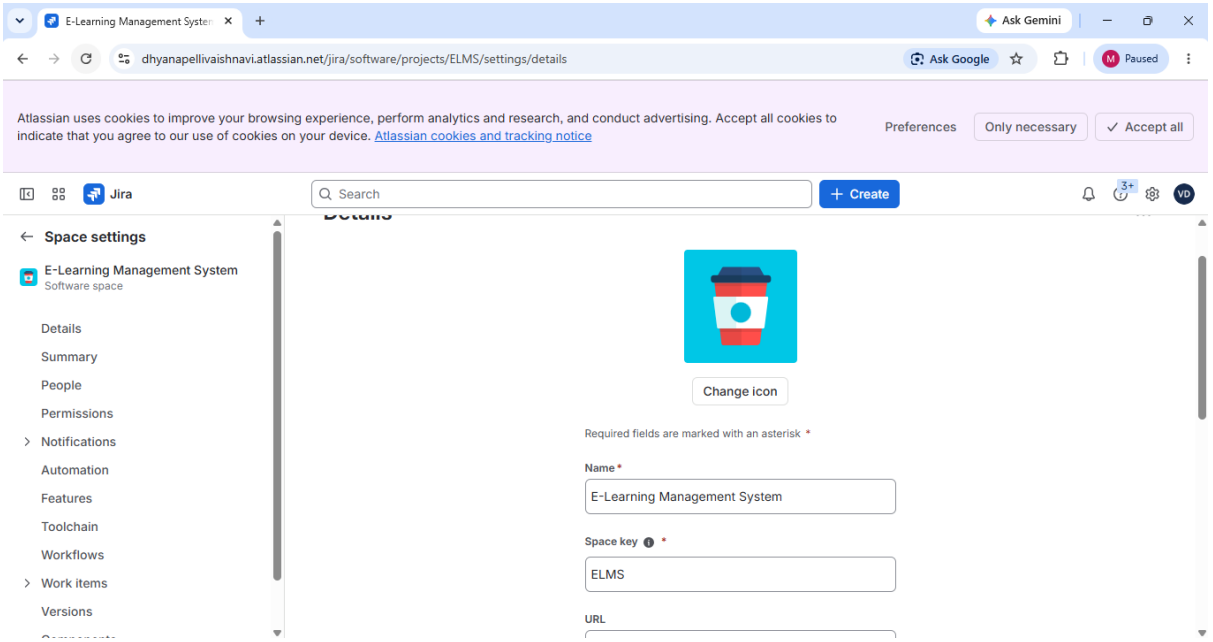
c) Configure Project Details.

Steps:-

1. Click on the Project Name and select the three-dot (⋮) menu.

2. Click on Space Settings.

3. Edit the Project Name as required.



4. Click on Save.

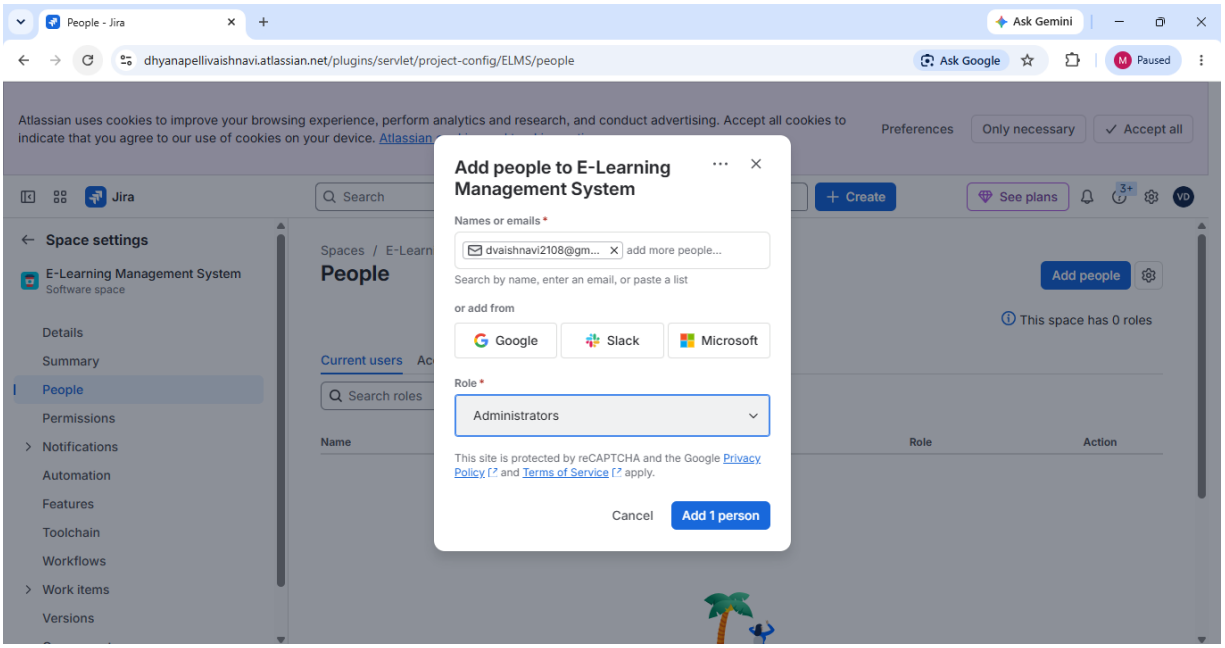
Practical No. 2

Aim:- Team Creation and Role Assignment.

a) Add Team Members

Steps:-

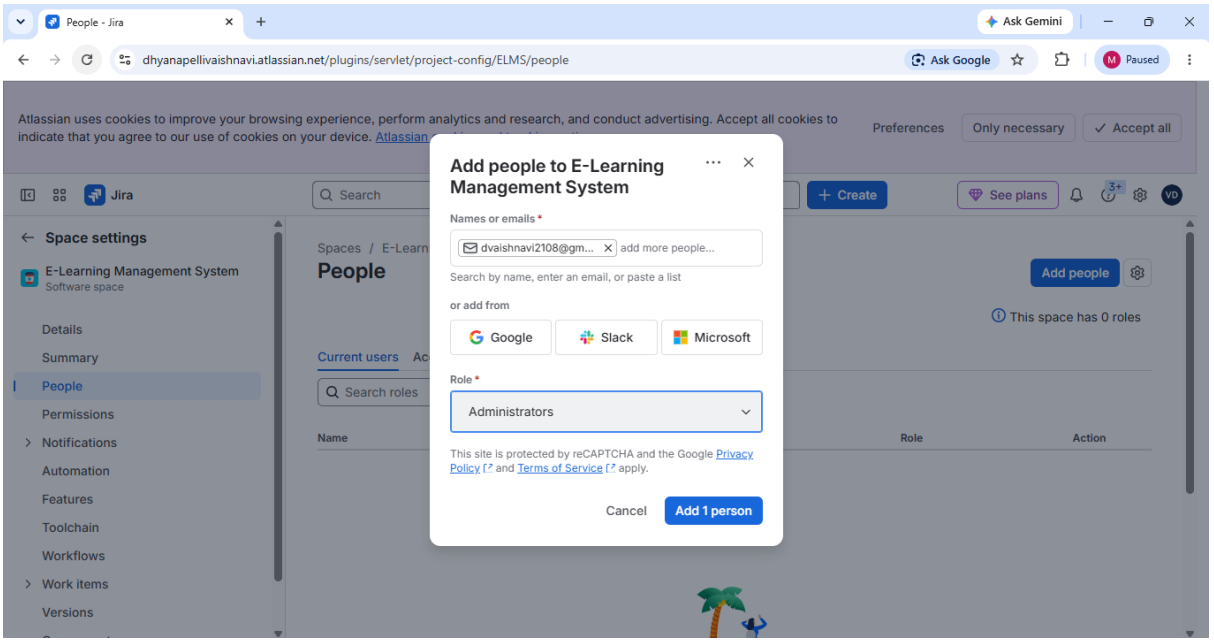
1. Open the E-Learning Management Space project.
2. Go to Space settings → People.
3. Click Add people.
4. Enter the team member's name or email.
5. Select the required role → Add
6. Verify that the member is added to the team.



b) Assign Roles

Steps:-

1. Open Space settings → People.
2. Find the required team member.
3. Click the Role option.
4. Select Administrator.
5. Save the changes.
6. Verify that the member has the Administrator role.



Practical No. 3

Aim:- Issue types and Configuration.

a) Create and Configure Issue Types (Epic, Story, Task, and Bug)

Steps:

1. Open the ELMS Dashboard.

2. Go to Backlog and click on Create.

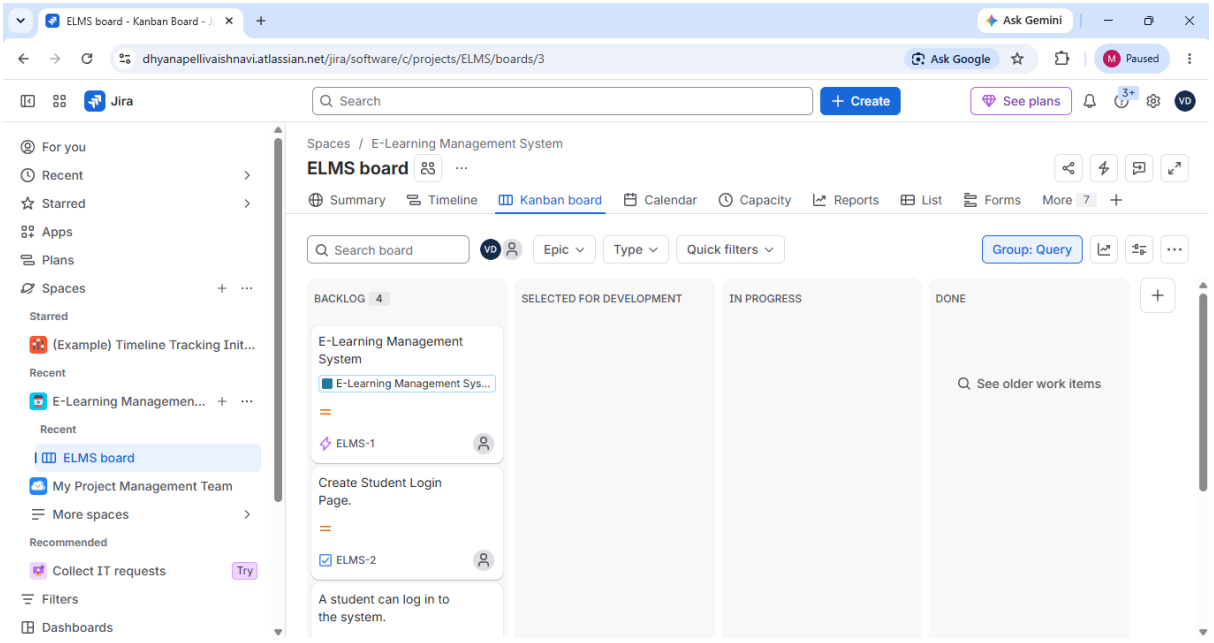
3. Create the following backlog items:

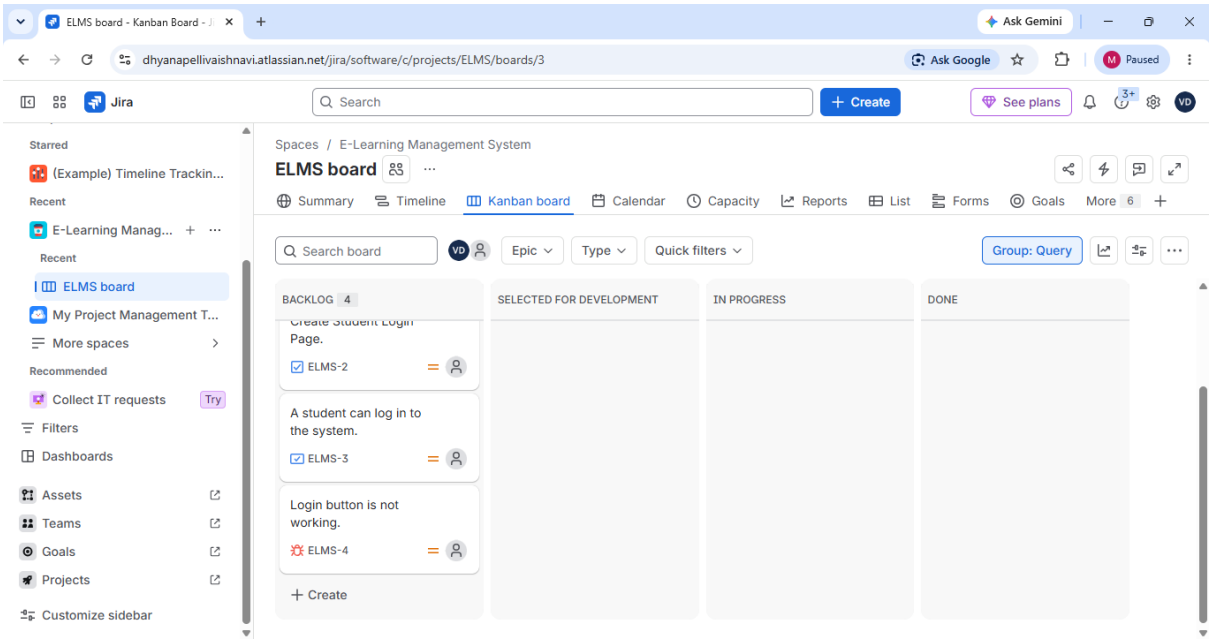
   a) Epic: E-Learning Management System

   b) Task: Create Student Login Page.

   c) Story: A student can log in to the system.

   d) Bug: Login button is not working.




ELMS board - Kanban Board -
Ask Gemini
dhyanapellivaishnavi.atlassian.net/jira/software/c/projects/ELMS/boards/3
Ask Google
Paused
Jira
Search
Create
See plans
Starred
(Example) Timeline Trackin...
Recent
E-Learning Manag...
Recent
ELMS board
My Project Management T...
More spaces
Recommended
Collect IT requests
Try
Filters
Dashboards
Assets
Teams
Goals
Projects
Customize sidebar
Spaces / E-Learning Management System
ELMS board
Summary
Timeline
Kanban board
Calendar
Capacity
Reports
List
Forms
Goals
More 6
Search board
Epic
Type
Quick filters
Group: Query
BACKLOG 4
SELECTED FOR DEVELOPMENT
IN PROGRESS
DONE
Page.
ELMS-2
A student can log in to the system.
ELMS-3
Login button is not working.
ELMS-4
Create

b) Create Components and Versions

Steps:

1. Go to Space Settings.

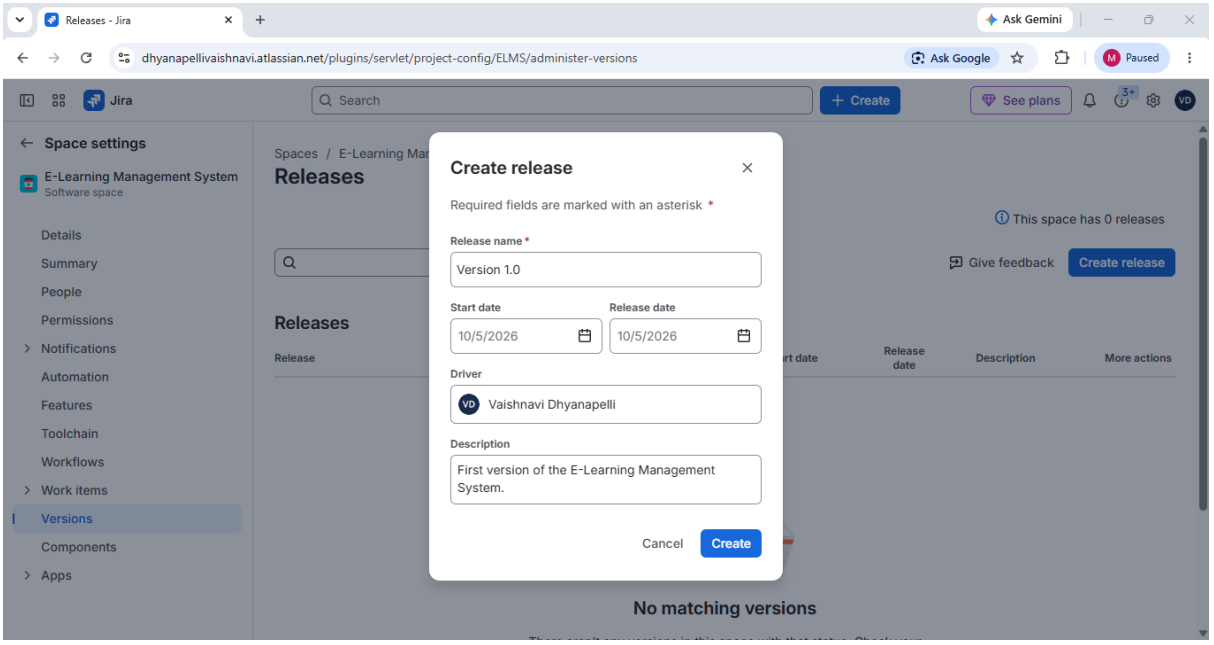
2. Click on Versions.

3. Click on Create Release.

4. Enter the following details:

   a) Release Name: Version 1.0

   b) Description: First version of the E-Learning Management System.



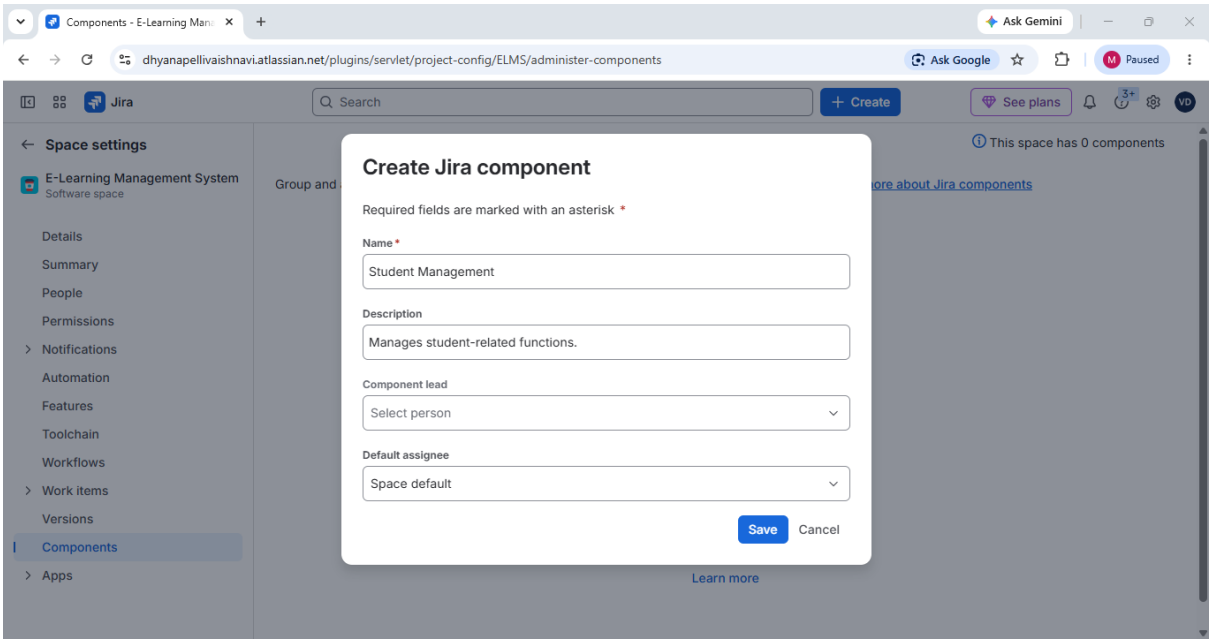
5. Click on Create.

6. Click on Components.

7. Create the first component with the following details:

   a) Name: Student Management

   b) Description: Manages student-related functions.



8. Click on Save.

9. Create the second component with the following details:

   a) Name: Teacher Management

   b) Description: Manages teacher-related functions.

10. Click on Save.

11. Create the third component with the following details:

   a) Name: Course Management

   b) Description: Manages course-related functions.

12. Click on Save.

Practical No. 4

Aim:- Issue Management.

a) Create an Issue Under an Epic

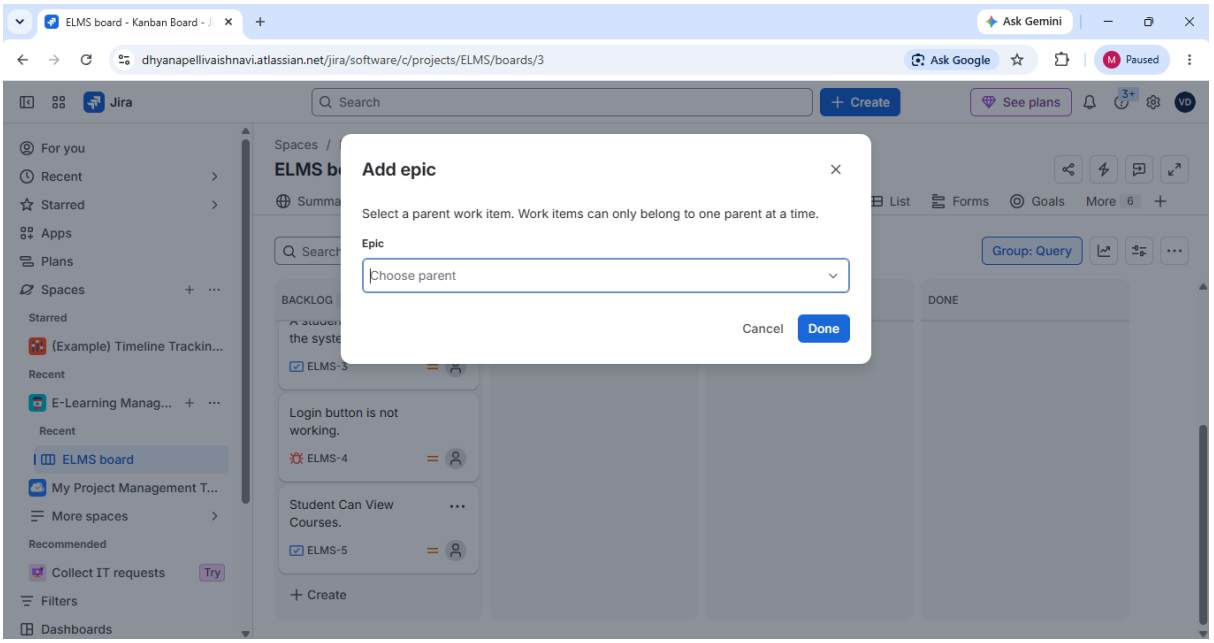
Steps:

1. Add a Story to the Backlog.

2. Enter the Story summary: Student Can View Courses.

3. Click on Add Parent.

4. Select the Epic ELMS (E-Learning Management System).



5. Click on Done.

b) Assign an Issue to a User and Set Priority and Due Date

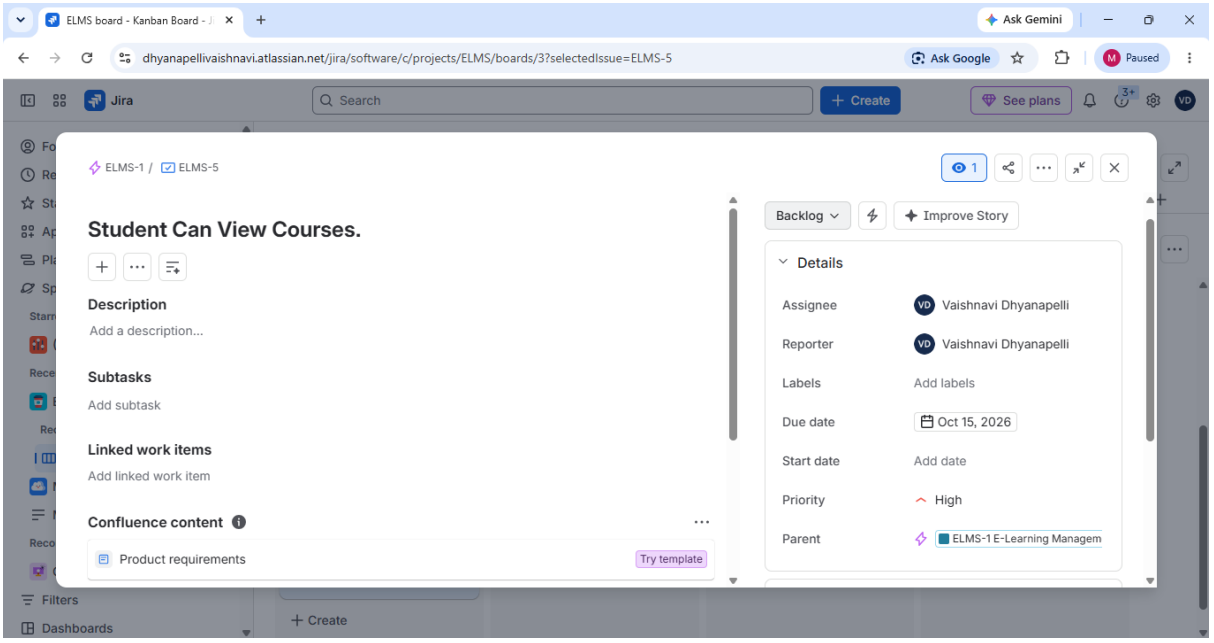
Steps:

1. Go to the Backlog and select the issue Student Can View Courses.

2. Assignee: Select Assignee.

3. Priority: Set the priority to High.

4. Due Date: Set the due date to 15/10/2026.



Practical No. 5

Aim:- Workflow Customization.

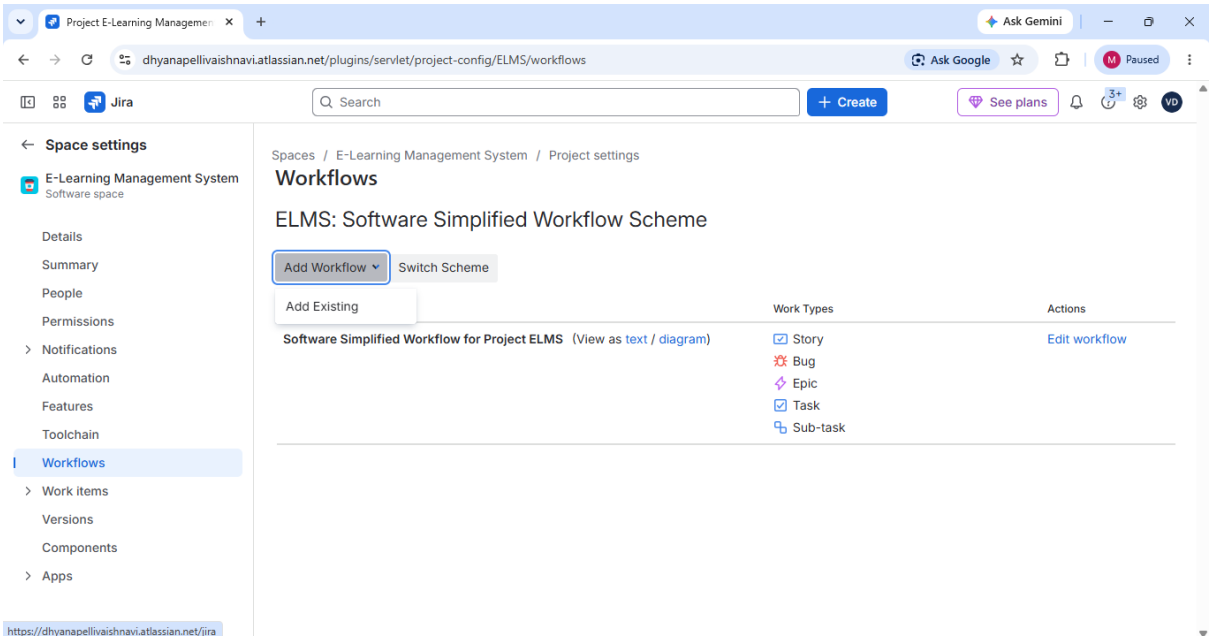
a) Design a Custom Workflow

Steps:

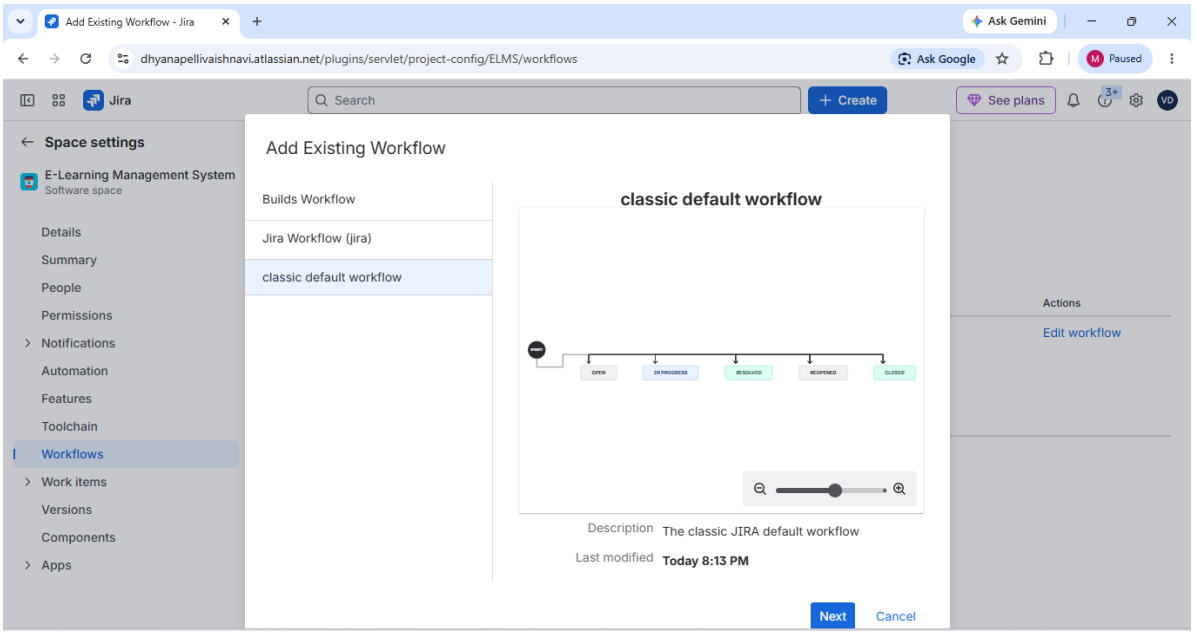
1. Go to Space Settings.

2. Click on Add Workflow.

3. Select Add Existing Workflow.

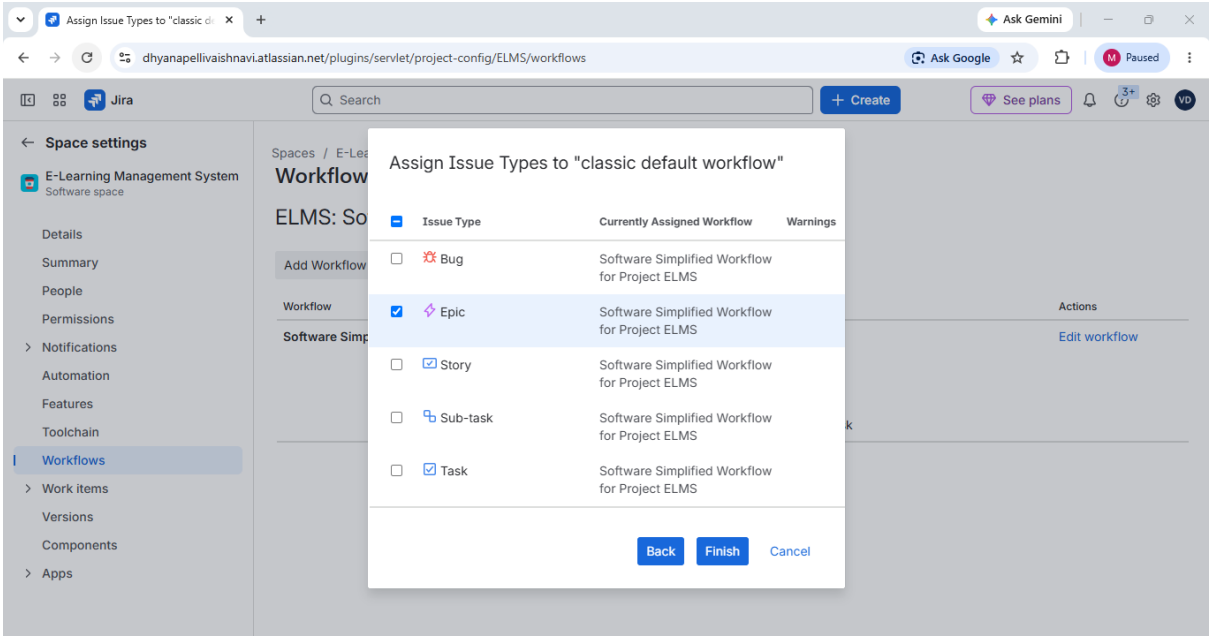


4. Add the Classic Default Workflow.



5. Click on Next.

6. Choose Epic as the issue type.



7. Click on Finish.

b) Add Transitions and Conditions

Steps:

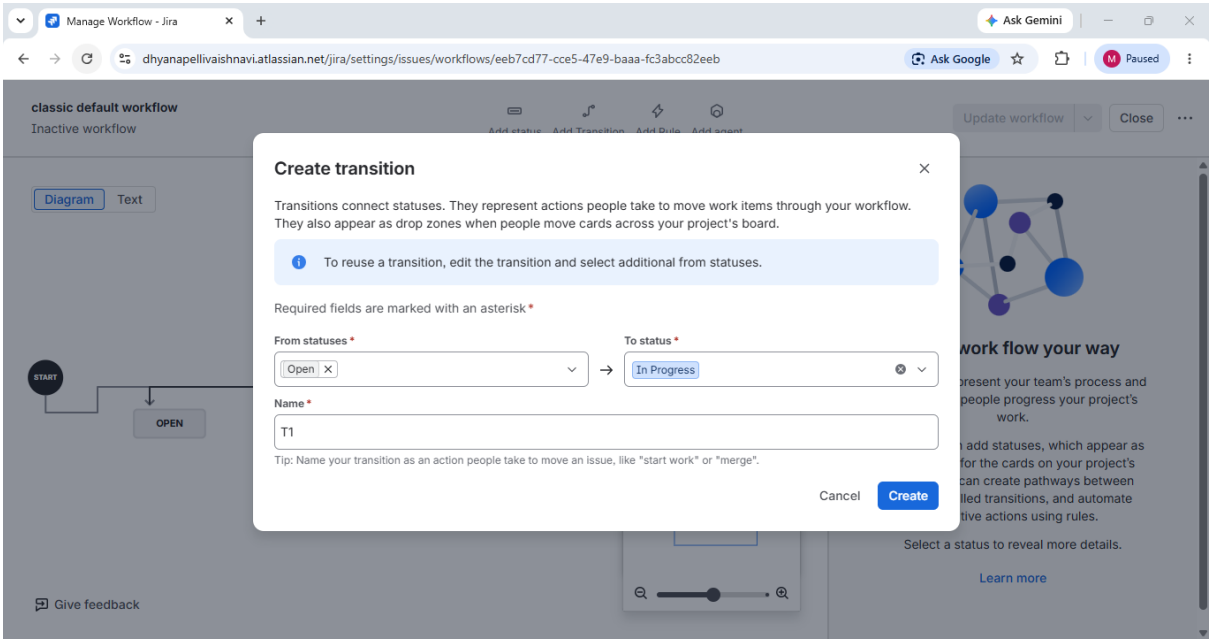
1. Open the Classic Default Workflow and click on Edit Workflow.

2. Click on Add Transition on the right side.

3. Set the From Status to Open.

4. Set the To Status to In Progress.

5. Enter the transition name as T1.



6. Click on Create.

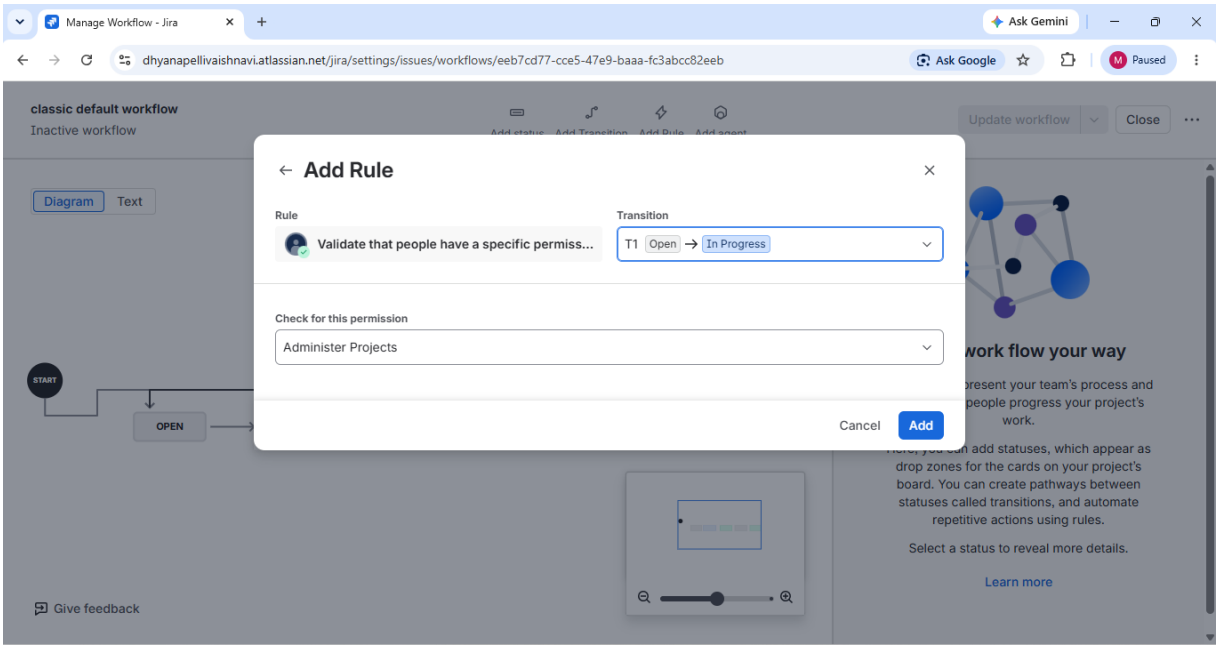
7. Click on Update Workflow at the top-right corner.

8. On the right side, click on Add Rule.

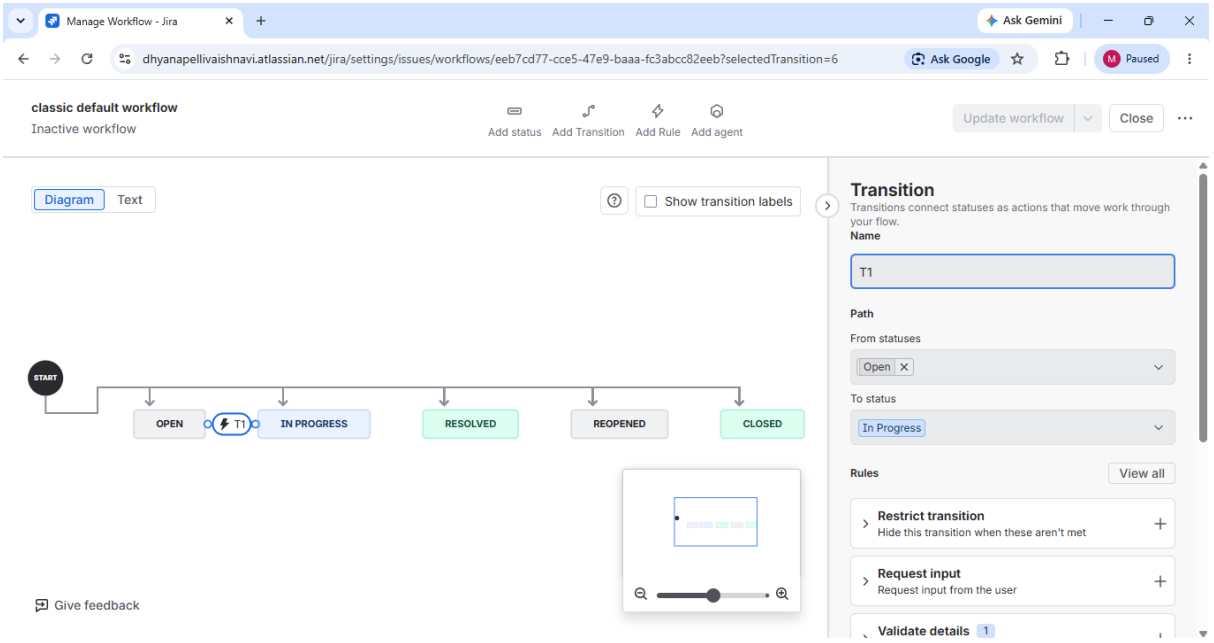
9. Select Validate Details to validate the details on the workflow.

10. Choose Validate that People have a Specific Permission.

11. Select Administrator Project.



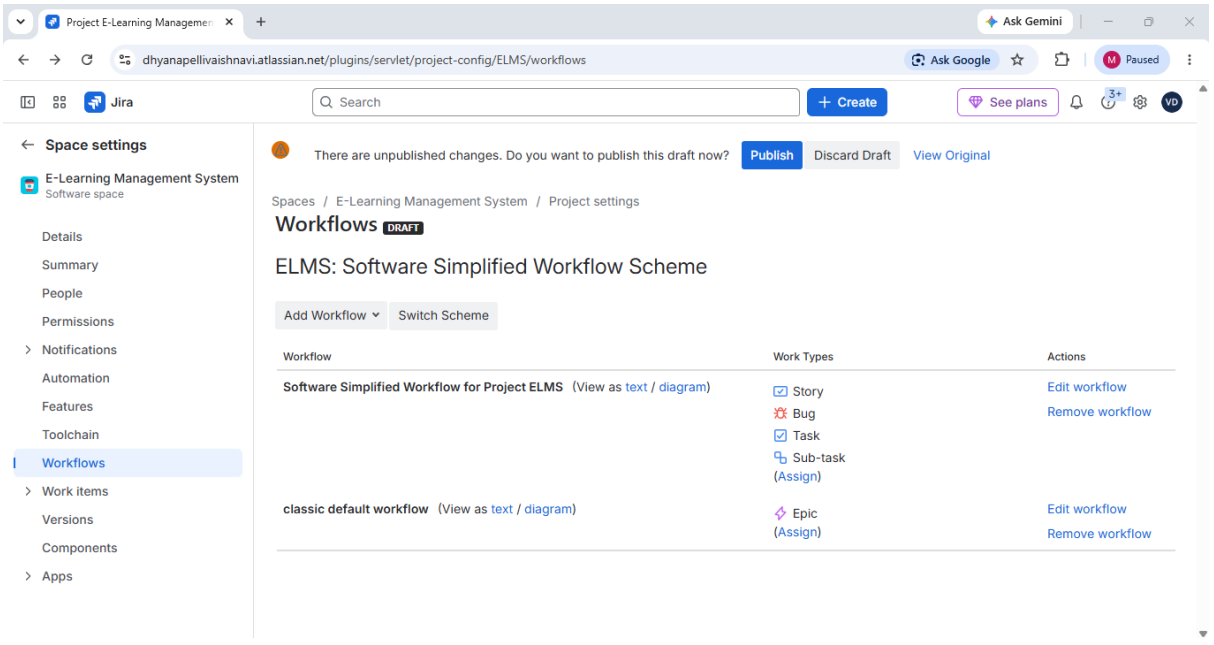
12. Click on Add and update the workflow.



c) Publish and Associate the Workflow with a Project

Steps:

1. Go back to the Dashboard.



2. Click on Publish.

3. Click on Associate.

4. Click on Acknowledge.

5. The workflow is successfully published and associated with the project.

Practical No. 6

Aim:- Scrum Board & Sprint Management.

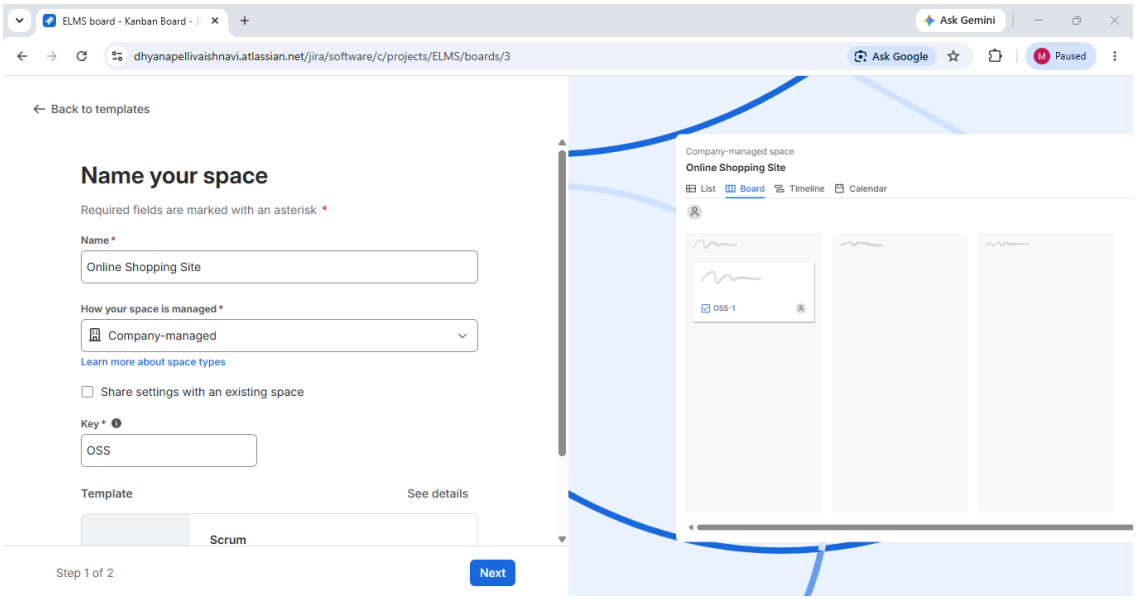
a) Create and Configure a Scrum Board

Steps:

1. Click on Spaces (+ icon) to create a new space.

2. Select Scrum and click on Use Template, then provide the required details.

3. Select Company-Managed in the space management option.



4. Click Next.

b) Create and Start a Sprint

Steps:

Create Sprint 1.

1. Select the issue type as Story.

2. Create the following user stories and assign story points:

   a) User Registration

   b) User Login

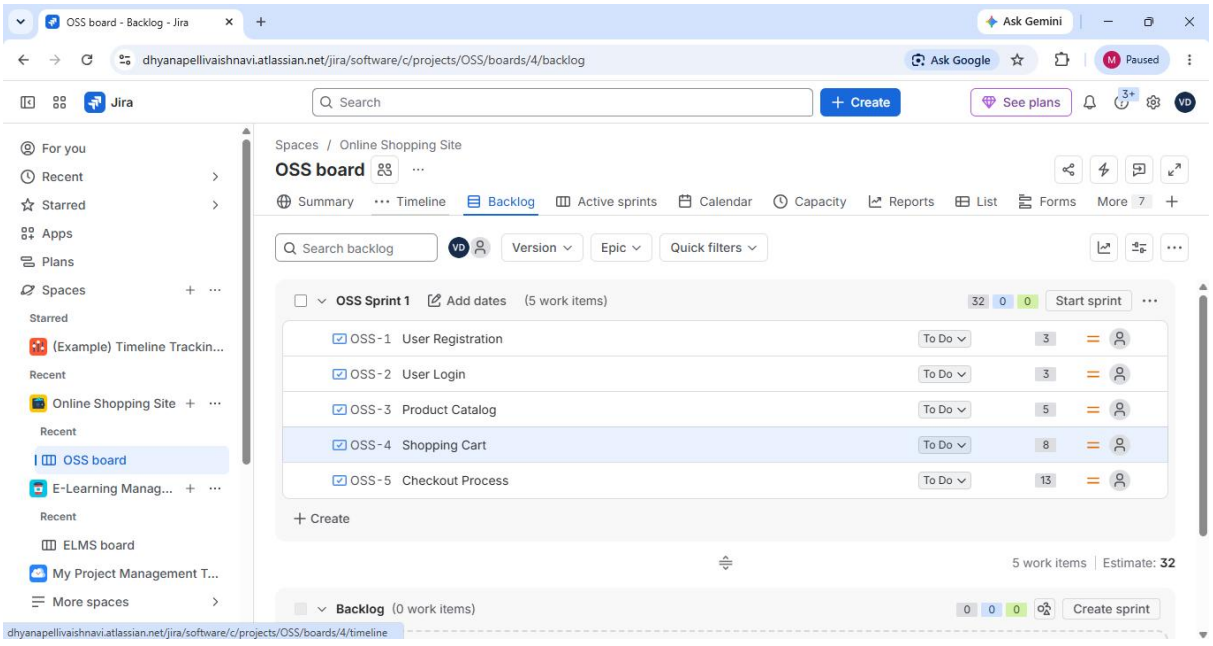
   c) Product Catalog

   d) Shopping Cart

   e) Checkout Process

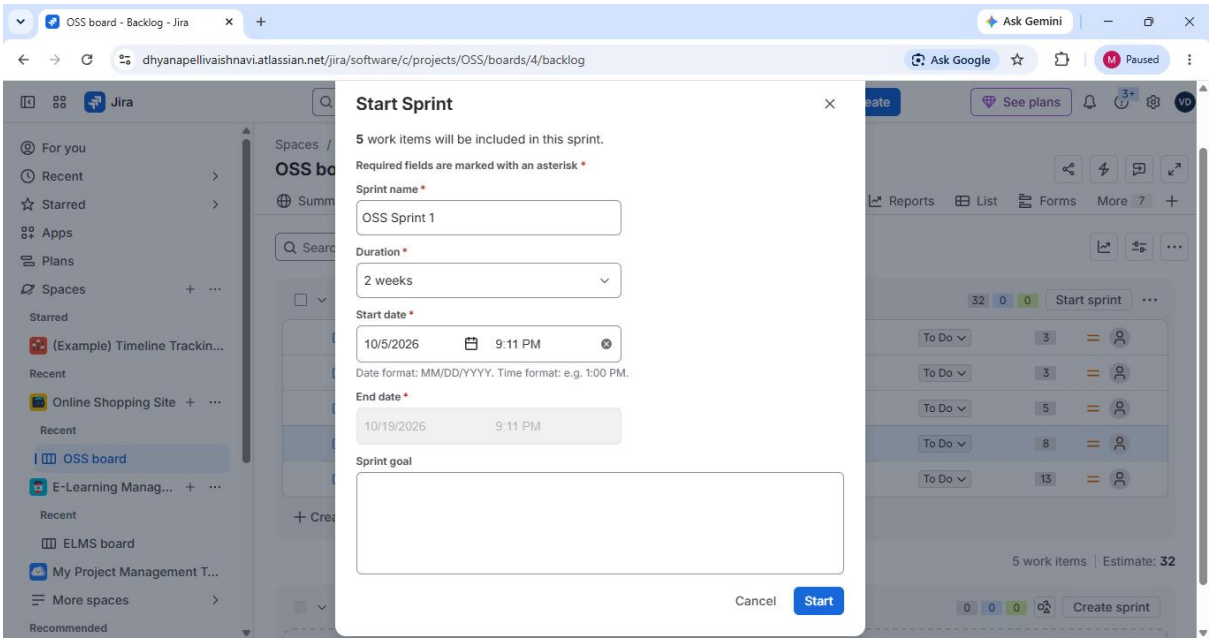
3. Add all the stories to Sprint 1 in the Backlog.

4. Add story points to each story.



5. Click on Start Sprint.

6. Enter the required sprint details and confirm by clicking on Start.

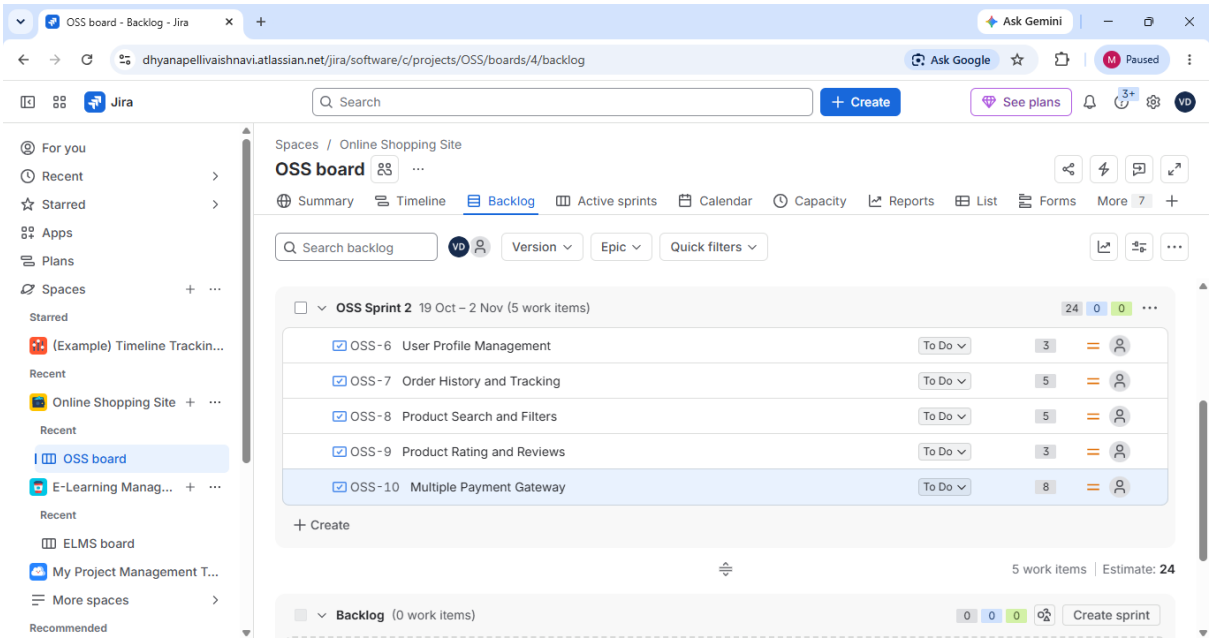


7. A confirmation message will appear, indicating that the sprint has started successfully.

Create Sprint 2

Steps:

1. Go to the Backlog.
2. Click on Create Sprint.
3. Set the issue type to Story.
4. Create the following user stories and assign story points:
   a) User Profile Management
   b) Order History and Tracking
   c) Product Search and Filters
   d) Product Rating and Reviews
   e) Multiple Payment Gateway
5. Add the stories to Sprint 2.


OSS board - Backlog - Jira
Ask Gemini
dhyanapellivaishnavi.atlassian.net/jira/software/c/projects/OSS/boards/4/backlog
Ask Google
Paused
Jira
Search
Create
See plans
For you
Recent
Starred
Apps
Plans
Spaces
Starred
(Example) Timeline Trackin...
Recent
Online Shopping Site
Recent
OSS board
E-Learning Manag...
Recent
ELMS board
My Project Management T...
More spaces
Recommended
Spaces / Online Shopping Site
OSS board
Summary
Timeline
Backlog
Active sprints
Calendar
Capacity
Reports
List
Forms
More 7
Search backlog
Version
Epic
Quick filters
OSS Sprint 2 19 Oct – 2 Nov (5 work items)
24 0 0
OSS-6 User Profile Management To Do 3
OSS-7 Order History and Tracking To Do 5
OSS-8 Product Search and Filters To Do 5
OSS-9 Product Rating and Reviews To Do 3
OSS-10 Multiple Payment Gateway To Do 8
Create
5 work items | Estimate: 24
Backlog (0 work items)
0 0 0
Create sprint

c) Complete a Sprint and Generate a Sprint Report

Steps:

1. Click on Summary.

2. Go to the Reports section and click on Sprint Report.

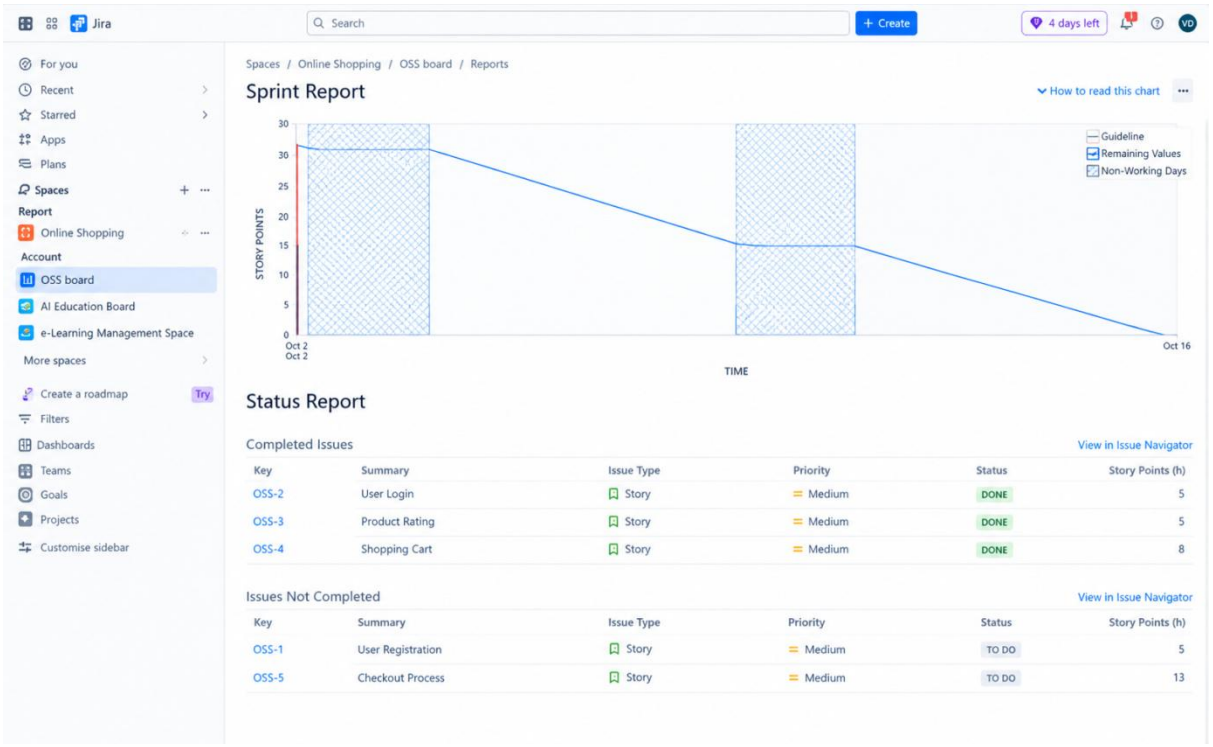
3. Return to the Backlog.

4. Complete all the issues in the sprint.

5. Return to Summary.

6. Open the Reports section.

7. The sprint report and graphs will be displayed.



Practical No. 7

Aim:- Kanban Board Management.

a) Create a Company-Managed Kanban Project.

Steps:

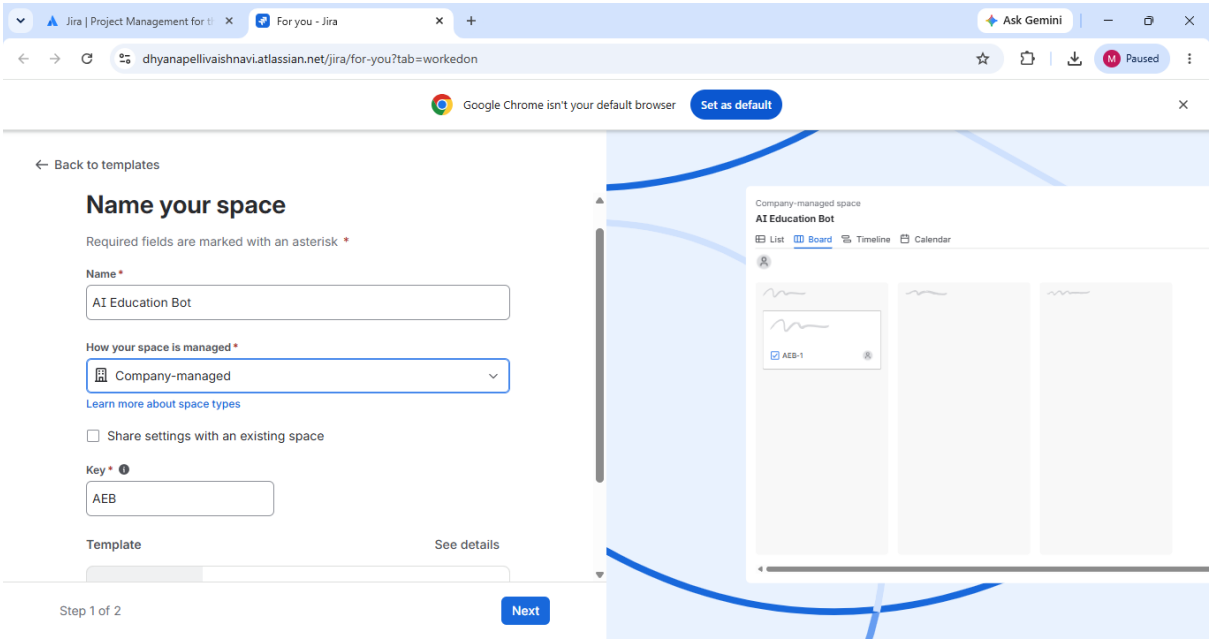
1. Go to Spaces and click on Create Space.

2. Select the Kanban template.

3. Use the selected template.

4. Enter the project name as AI Education Bot.

5. Enter the description as Company Managed.



6. Click on Next.

b) Configure the Kanban Board

Steps:

1. Go to the Backlog.

2. Create a task with the following details:

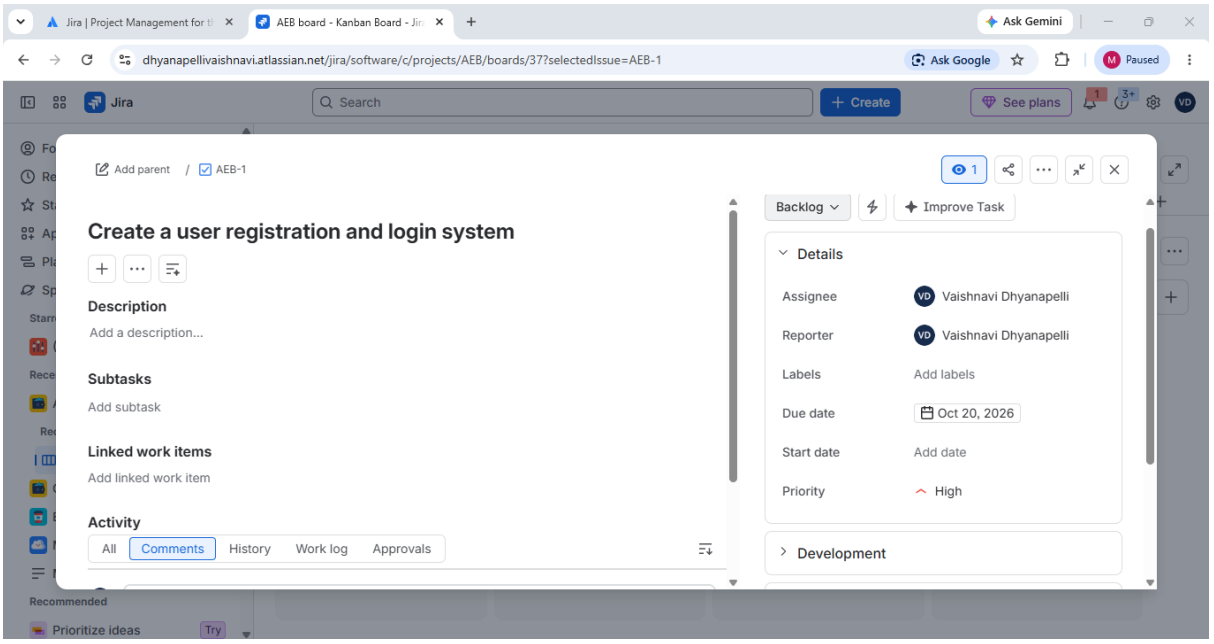
   - Issue Type: Task

   - Summary: Create a user registration and login system

   - Assignee: Assign to Me

   - Due Date: 20/09/2026

   - Priority: High



3. Open the In Progress column.

4. Add Developed Login API.

5. Change the issue type to Task.

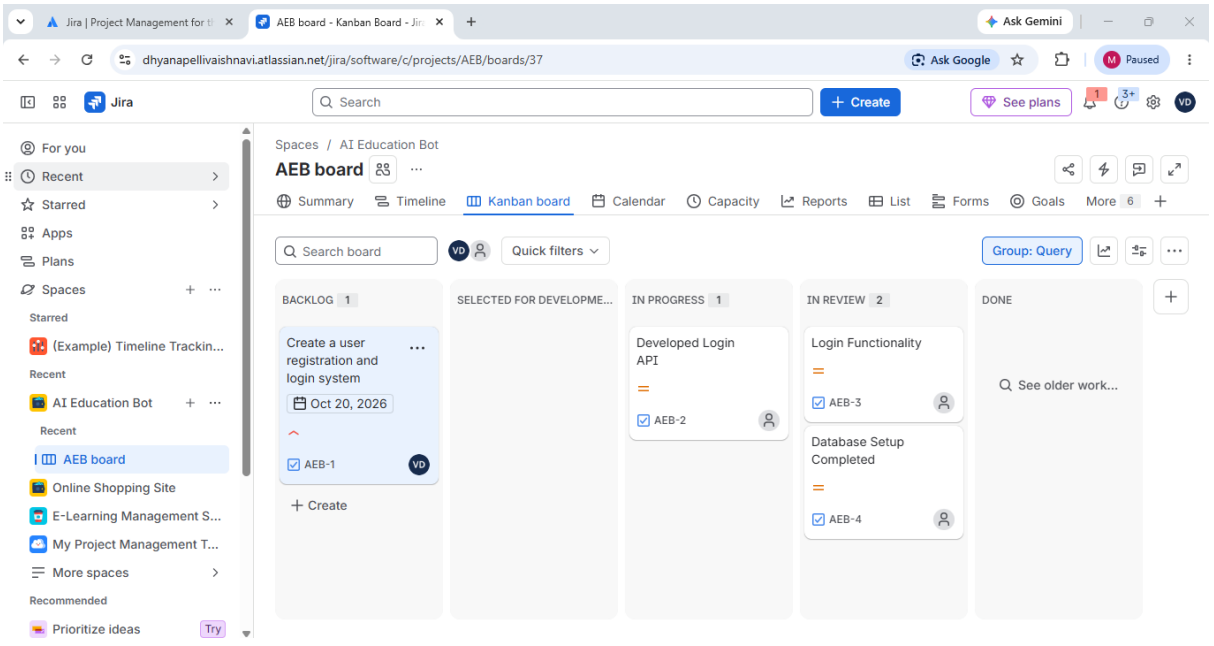
6. Create a new column named IN REVIEW.

7. IN REVIEW column.

8. Add Task:

    1. Login Functionality

    2. Database Setup Completed


Jira | Project Management for t
AEB board - Kanban Board - Jir
Ask Gemini
dhyanapellivaishnavi.atlassian.net/jira/software/c/projects/AEB/boards/37
Ask Google
Paused
Jira
Search
Create
See plans
For you
Recent
Starred
Apps
Plans
Spaces
Starred
(Example) Timeline Trackin...
Recent
AI Education Bot
Recent
AEB board
Online Shopping Site
E-Learning Management S...
My Project Management T...
More spaces
Recommended
Prioritize ideas
Try
Spaces / AI Education Bot
AEB board
Summary
Timeline
Kanban board
Calendar
Capacity
Reports
List
Forms
Goals
More 6
Search board
Quick filters
Group: Query
BACKLOG 1
Create a user registration and login system
Oct 20, 2026
AEB-1
Create
SELECTED FOR DEVELOPME...
IN PROGRESS 1
Developed Login API
AEB-2
IN REVIEW 2
Login Functionality
AEB-3
Database Setup Completed
AEB-4
DONE
See older work...

c) Track Issue Progress and Associate a Control Chart

Steps:

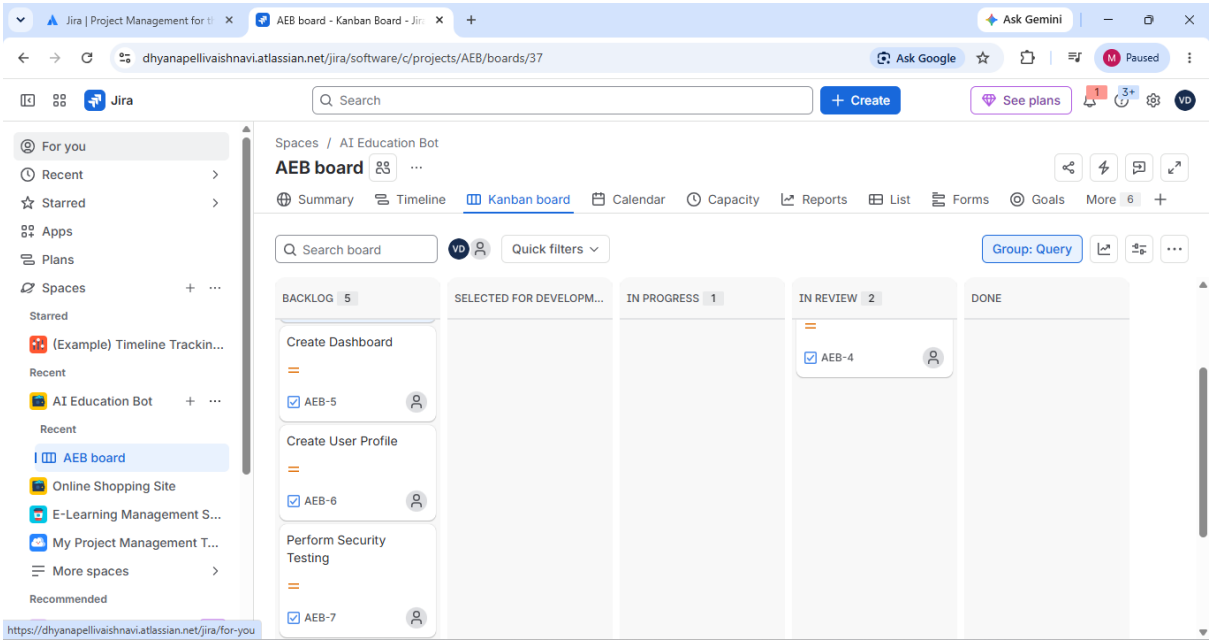
1. Go to Backlog Add this 4 Tasks in Backlog Column.

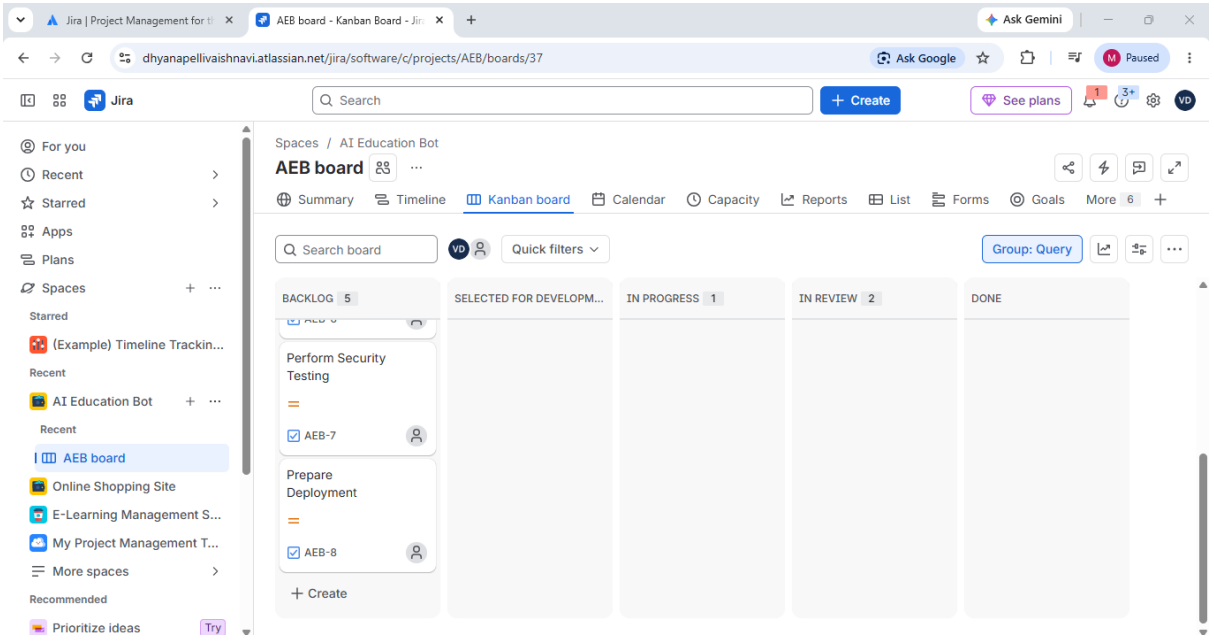
2. Create Dashboard.

3. Create User Profile.

4. Perform Security Testing.

5. Prepare Deployment.



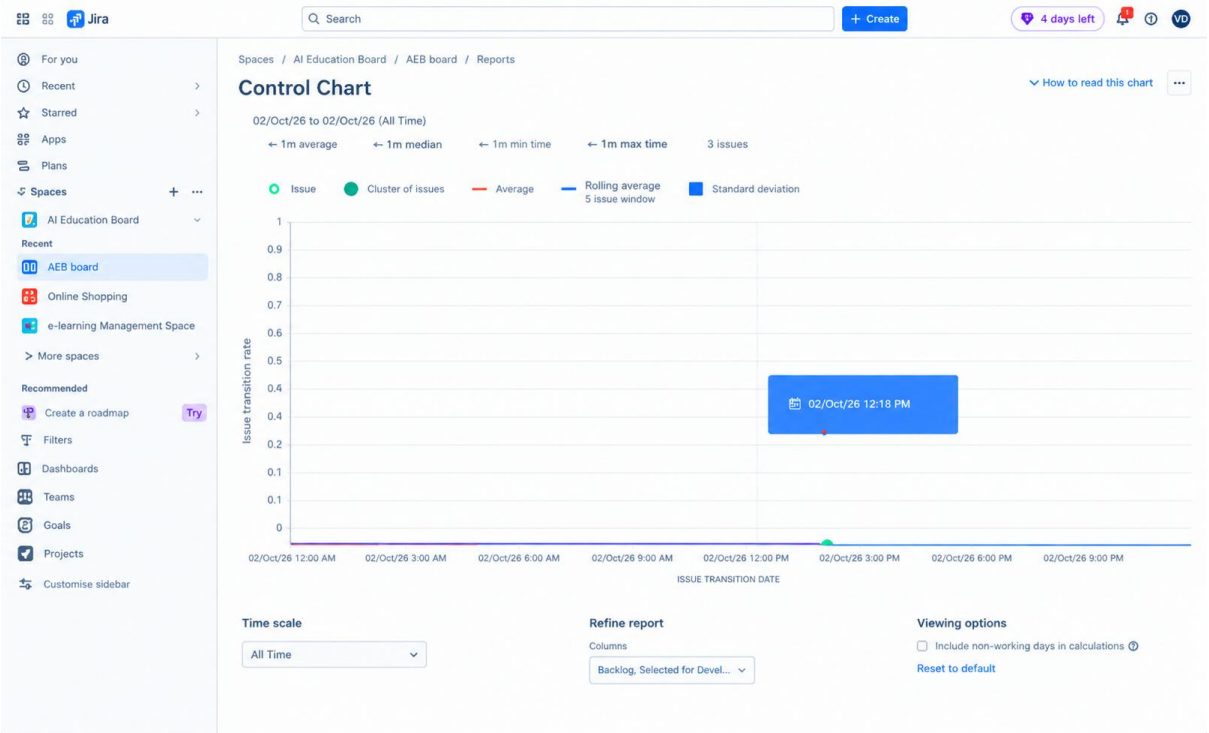


5. Open Reports.

6. Click on More Options and select the Control Chart.

7. In the Columns section, select the required columns.

8. The selected changes will be displayed in the chart.



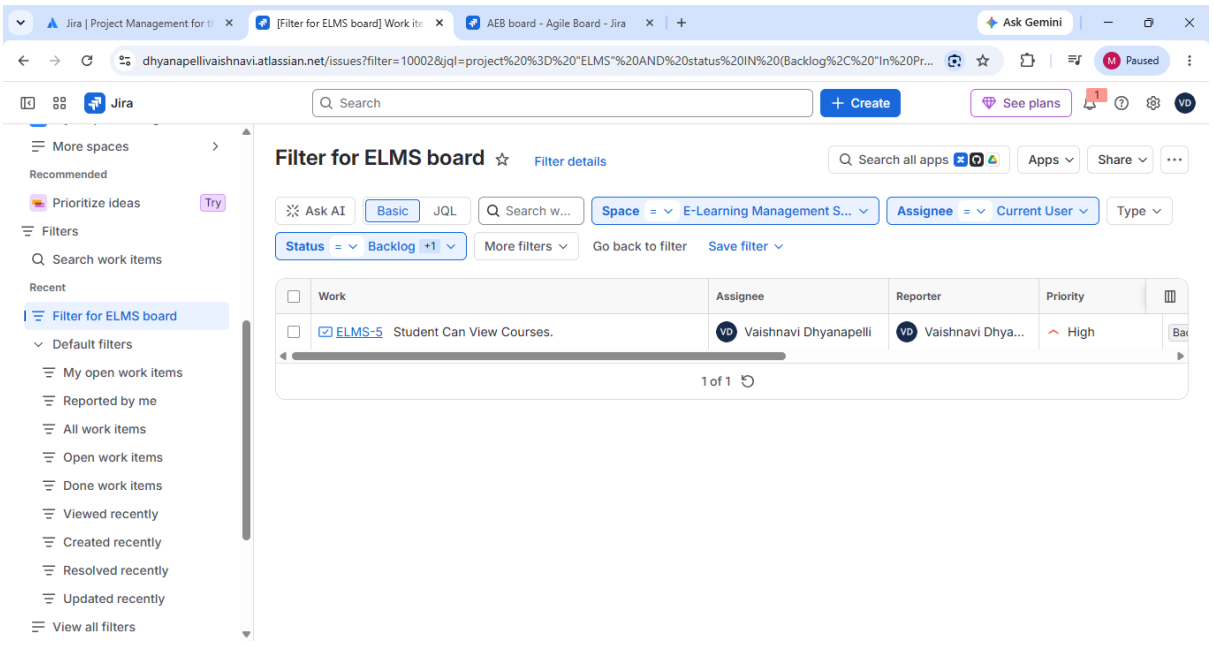
Practical No. 8

Aim:- JQL & Filters.

a) Write basic JQL queries (Status, Assignee, Priority)

Steps:

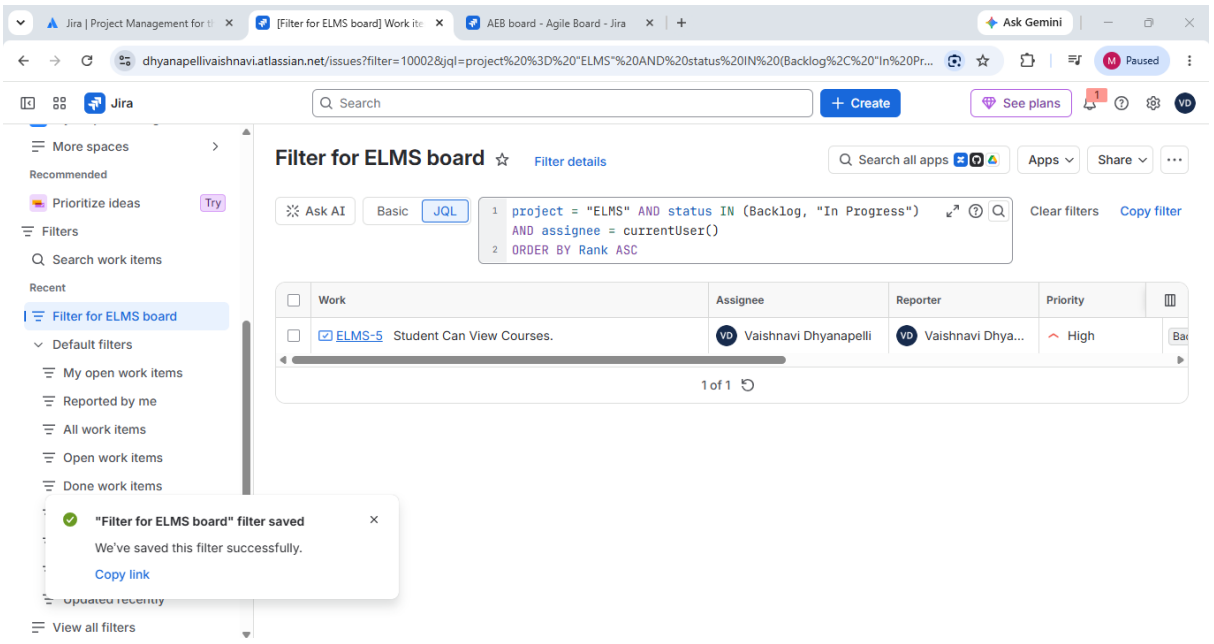
1. Go to Filters → View all filters.
2. Select Filters for E-LMS Board.
3. Set Status → Backlog & In Progress.
4. Set Assignee → Current User.
5. Set Priority → High.



b) Create and save filters

Steps:

1. Go to Filters → Advanced Issue Search.
2. Selected required Project, Status, Assignee and Priority.
3. Click Save as.
4. Click on JQL → See Query



Practical No. 9

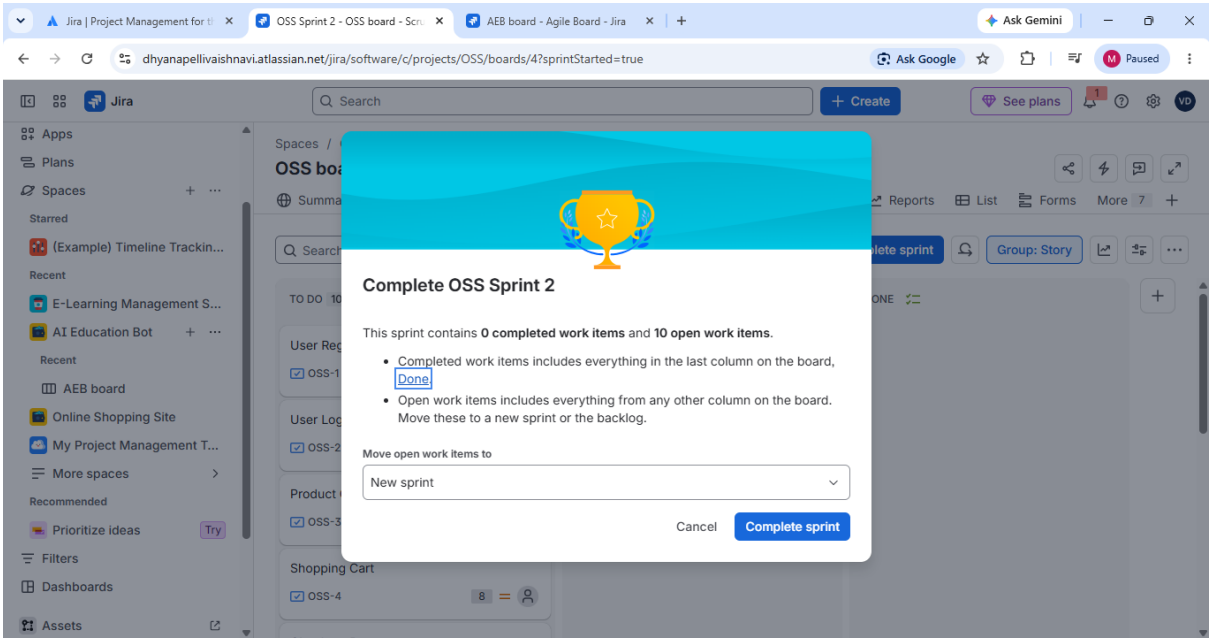
Aim:- Reports & Dashboards.

a) Generate Burndown Chart and Velocity Chart

Steps:

1. Open the Jira Dashboard.

2. Start the Sprint and complete the sprint.

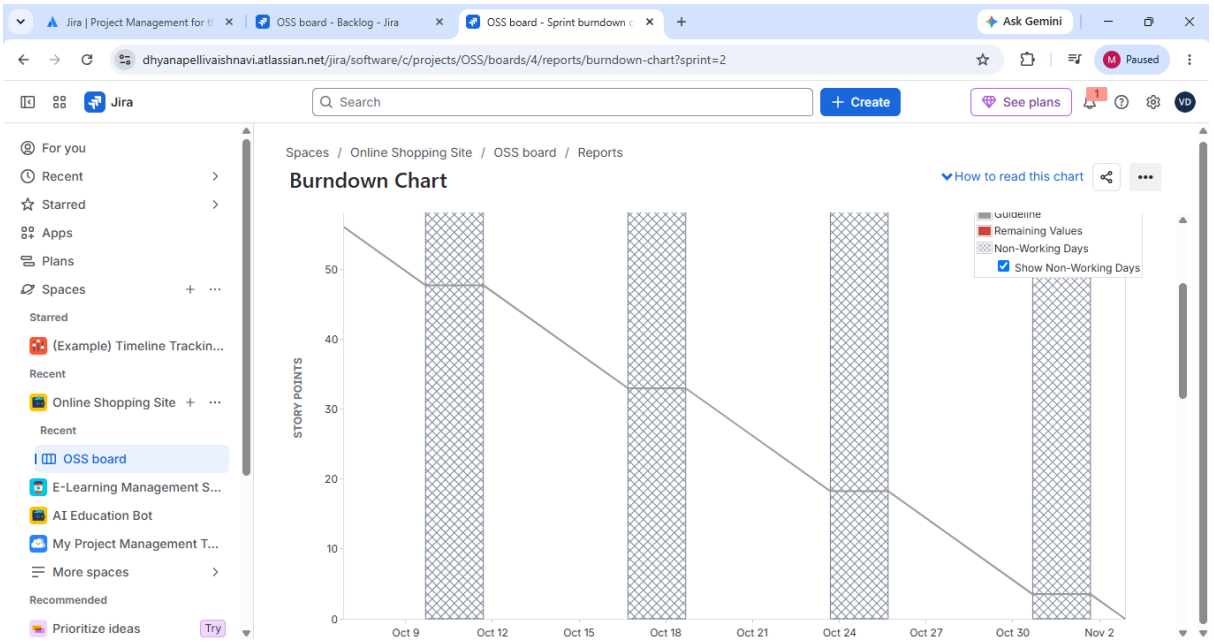


3. Go to Reports and click on More Reports.

Burndown Chart:

4. For the Burndown Chart:

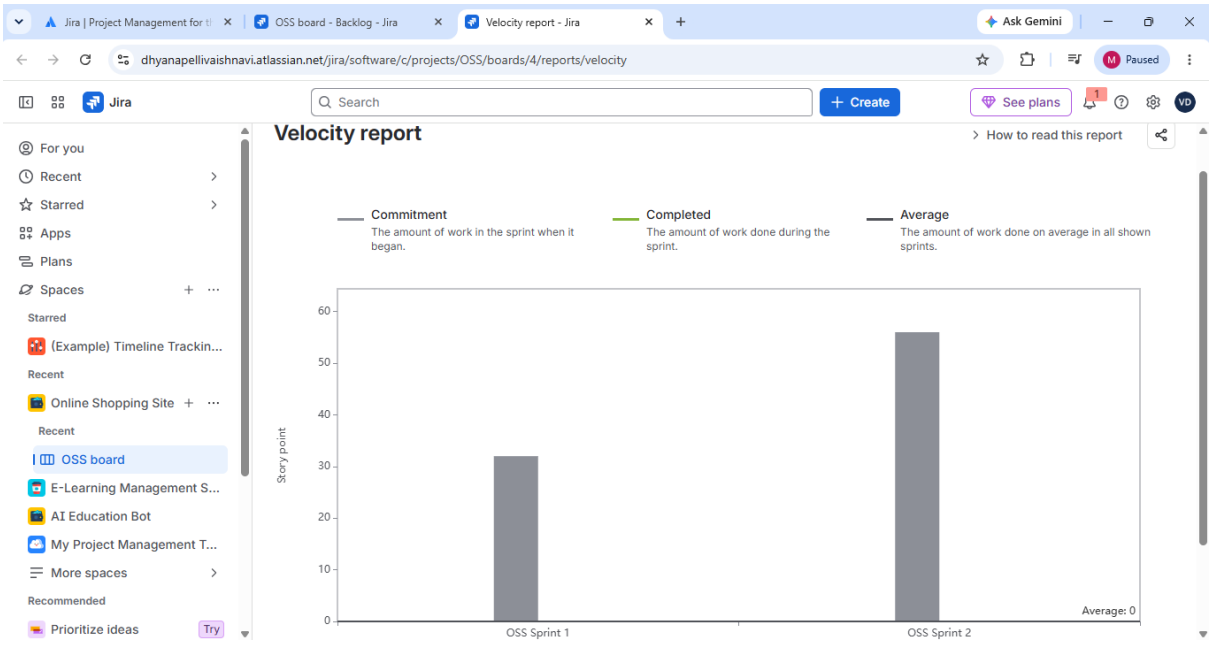
- Select the OSS Project.
- Click on Reports and select Burndown Chart.



Velocity Chart:

5. For the Velocity Chart:

- Select the OSS Project.
- Click on Reports and select Velocity Chart.



b) Create Pie Chart and Issue Statistics Reports

Steps:

For Pie Chart:

1. Select the project.

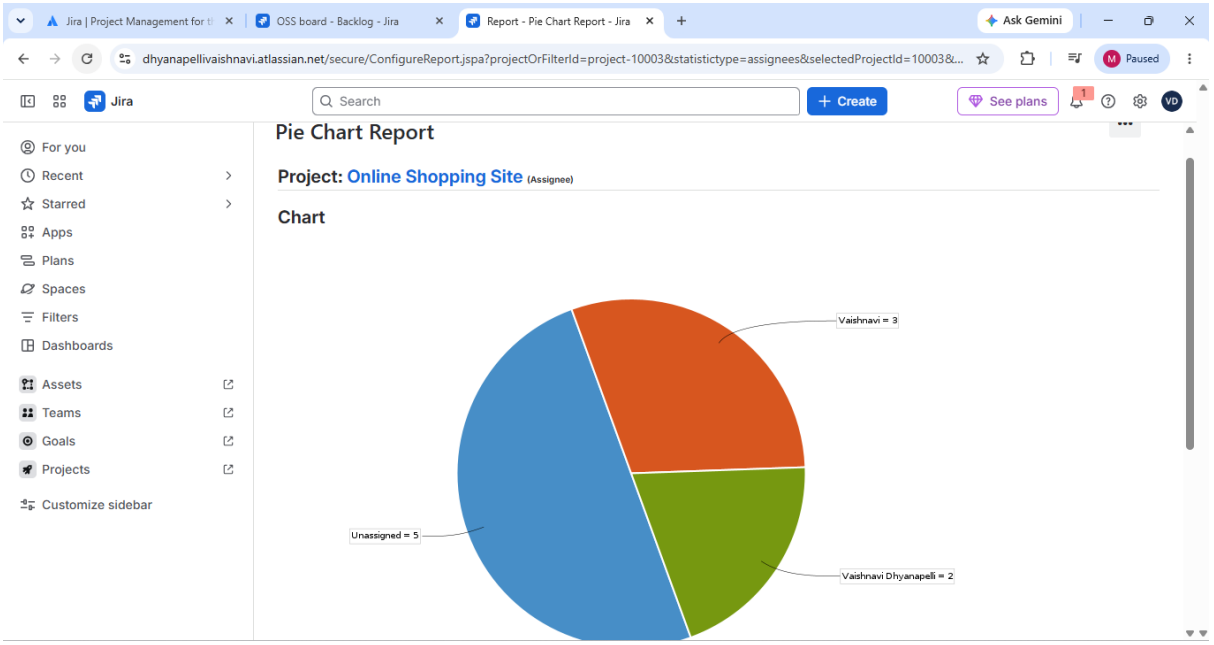
2. Assign work to the users.

3. Click on Reports and then click on More Reports.

4. Select Pie Chart.

5. Click on Next.

6. The Pie Chart will be generated and displayed.



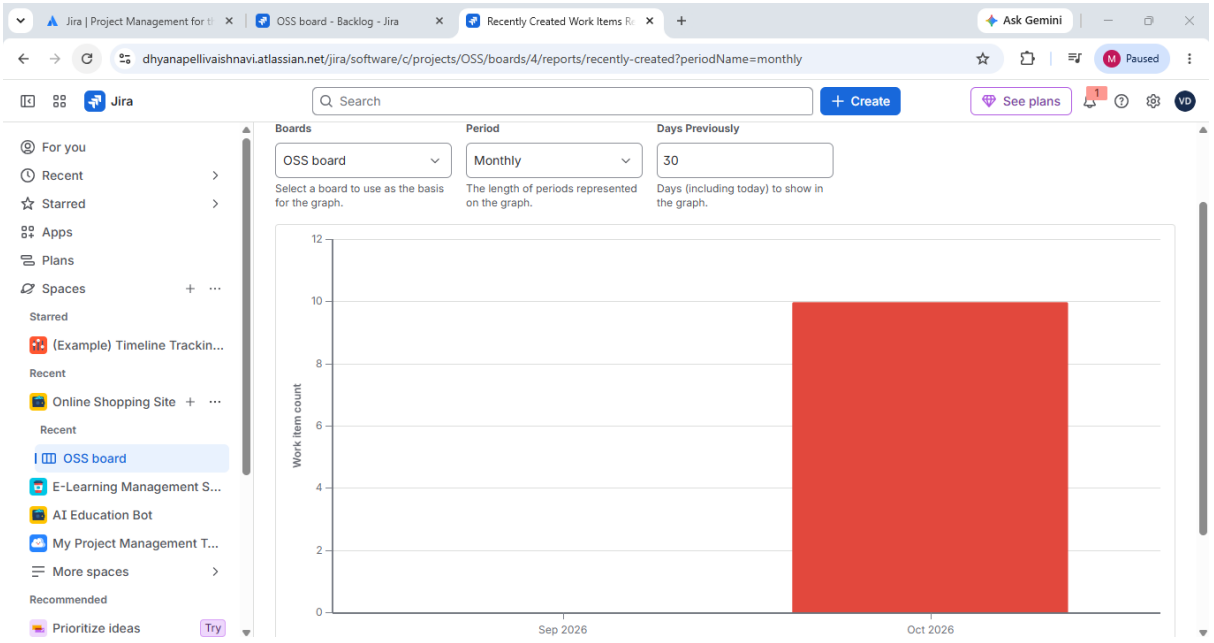
For Issue Statistics Report:

1. Click on Reports and then click on More Reports.

2. Select the recently created Work Issue Report.

3. Change the period from Daily to Monthly.

4. The report will be displayed.



c) Create a Custom Dashboard with Gadgets

Steps:

1. Click on Dashboard from the left panel.

2. Click on Create Dashboard.

3. Enter the name as Customize AEB Dashboard.

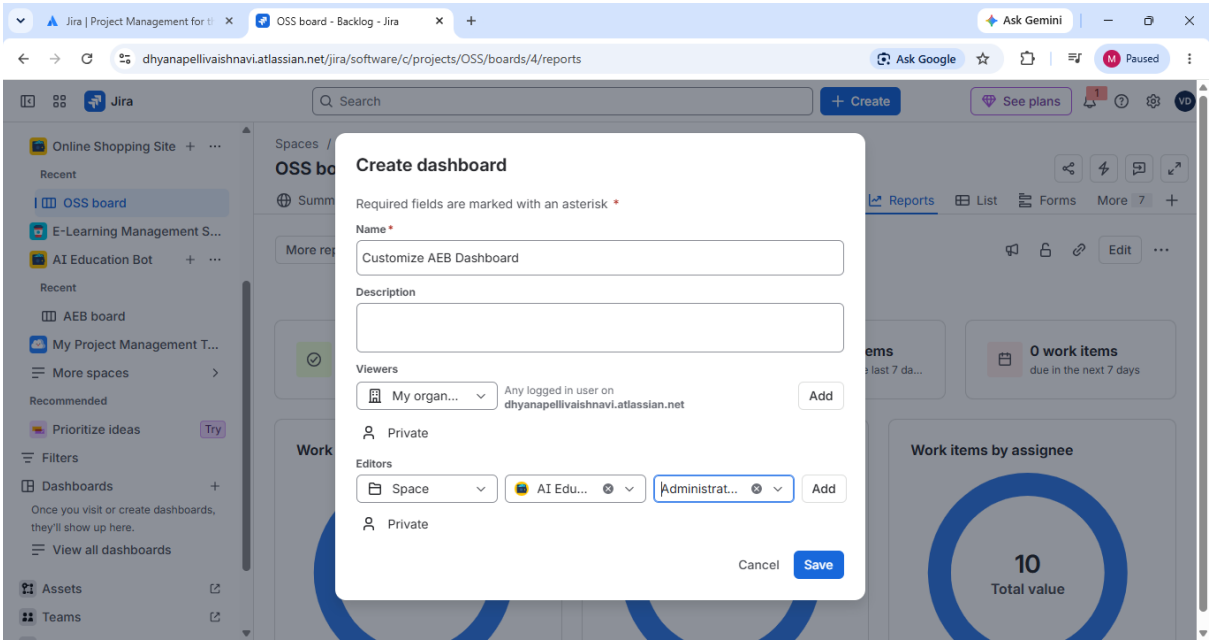
4. Under Viewers, select My Organization.

5. Under Editors, select space.

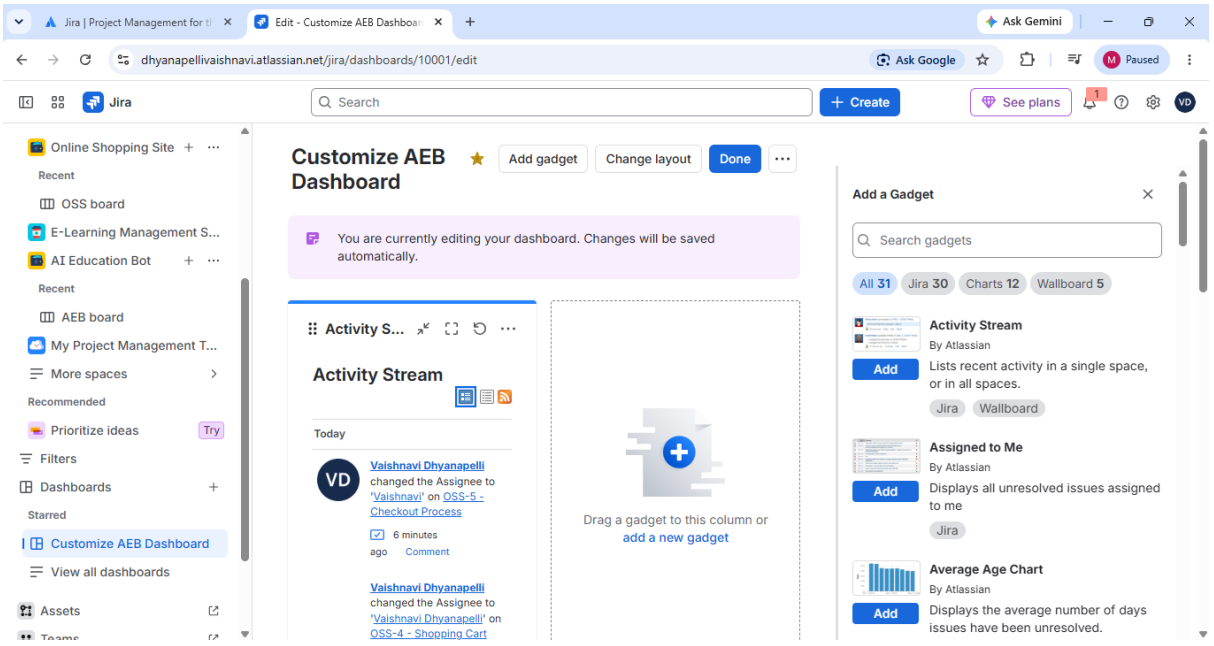
6. Select AI Education Bot.

7. Select Administrator.

8. Click on Add and Save.



9. Select Activity Stream.

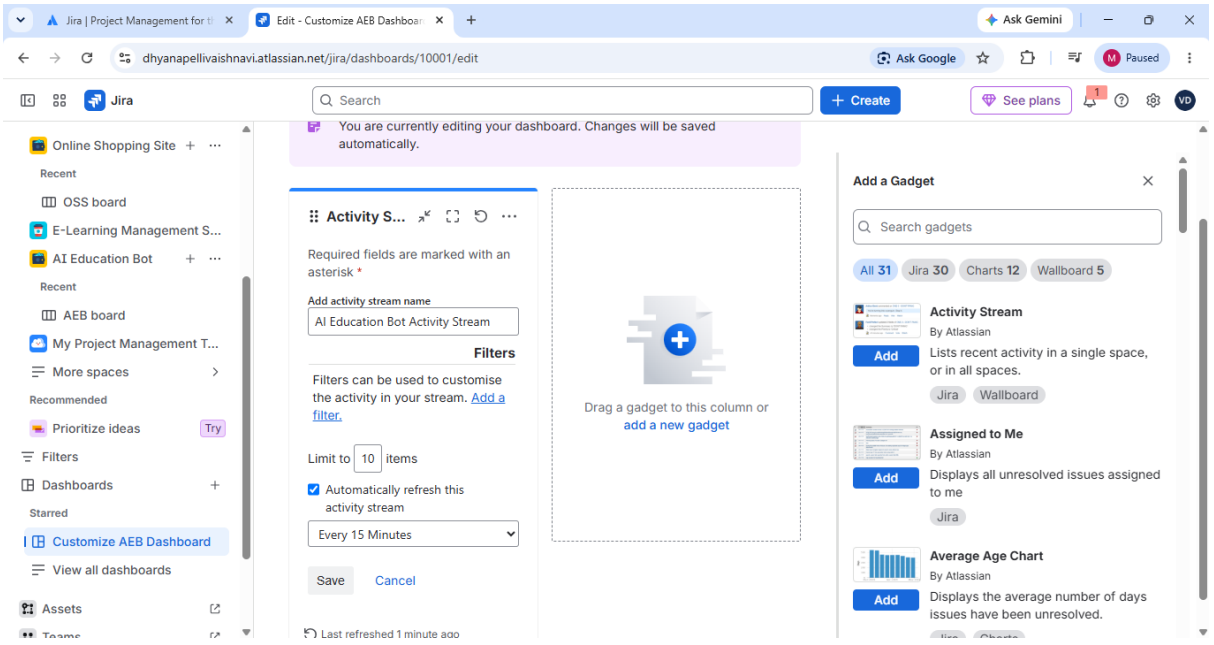


10. Click on the three dots and select Configure.

11. Set the Activity Stream to AI Education Bot Activity Stream.

12. Set the limit to 10 items.

13. Select the required checkbox.



14. Click on Save.

Practical No. 10

Aim:- Permissions & Security Management.

a) View and Modify Project Permission Schemes

Steps:

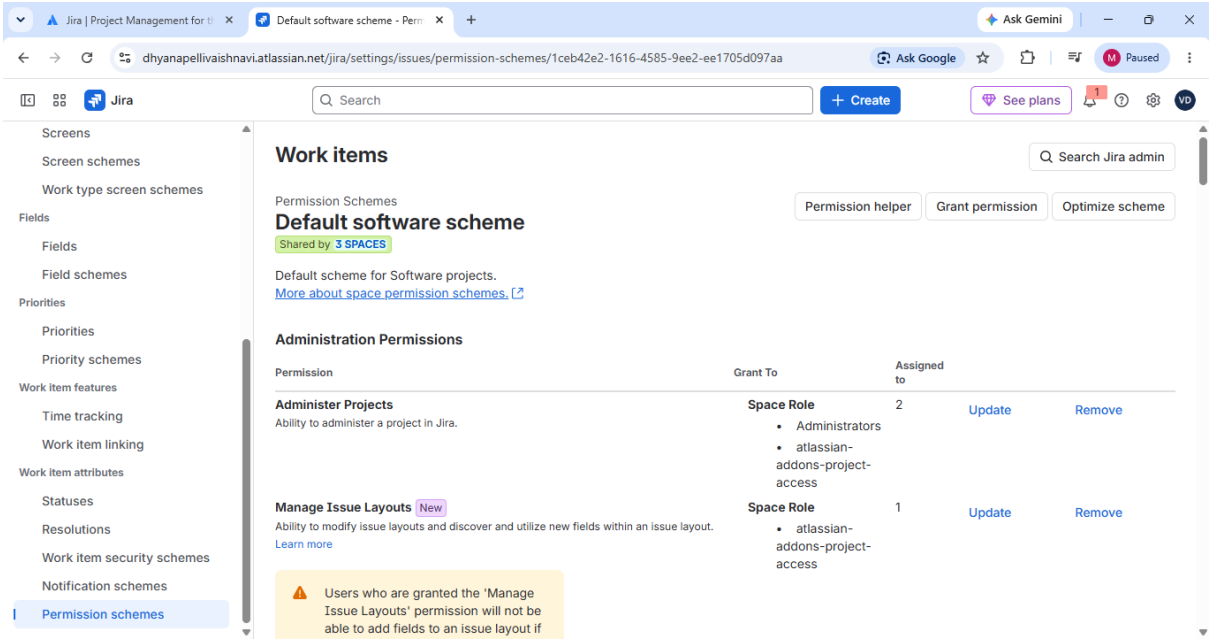
1. Select the Project.

2. Click on Space Settings.

3. Click on Permissions.

4. Click on Actions.

5. Click on Update and Remove to edit the permissions.



b) Configure Project Role and Assign Permission

Steps:

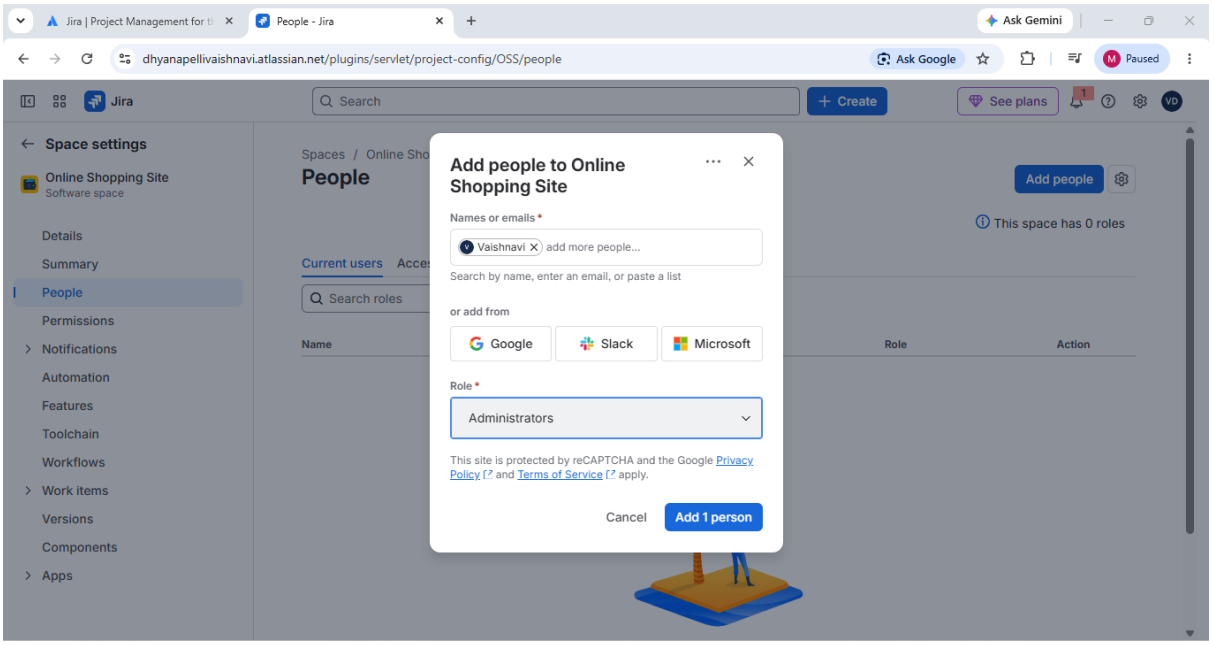
1. Select the Project.

2. Click on Space Settings.

3. Click on People.

4. Add a person and assign a suitable role.

5. Click on Add 1 Person.



Practical No. 11

Aim:- Jira Automation.

a) Create an Automation Rule (Trigger → Condition → Action)

Steps:

1. Open the OSS Board.

2. Click on Automation icon on the top right side.

3. Create Automation.

4. Click on Next 4 times.

5. Click on Get Started.

6. Click on X button.

7. Search for Search for Field Value Changed.

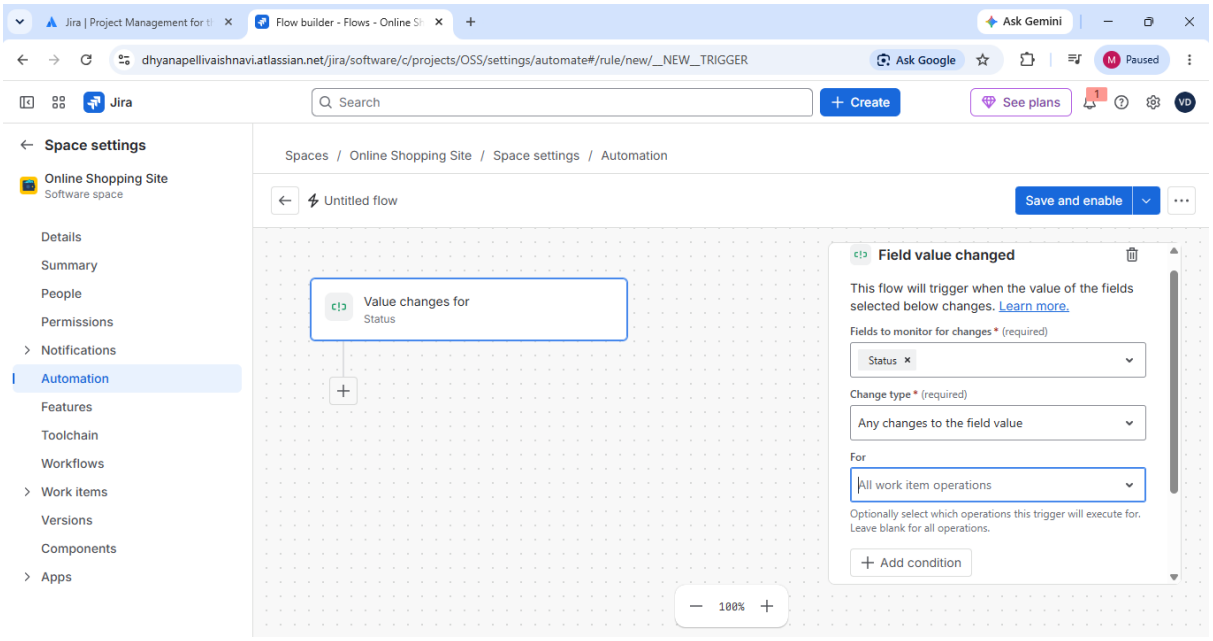
8. Click on the Field Value Changed option.

9. Change the field to Status.

10. Change type, Any Changes to the field value.

11. For, select All work item operations.

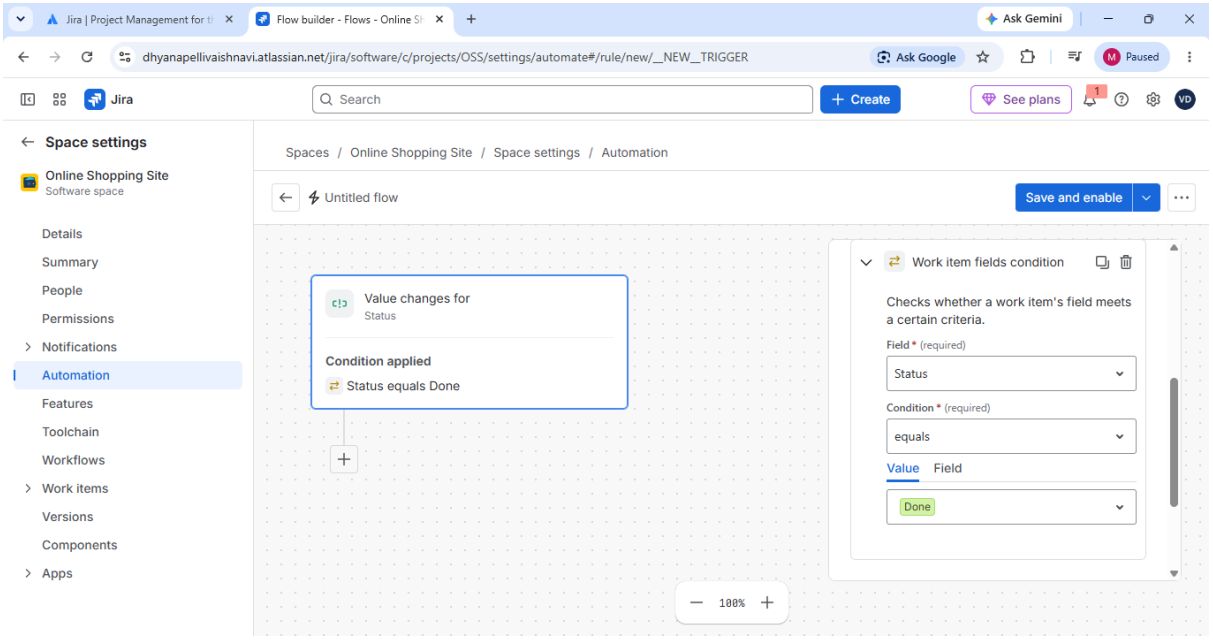
12. Click on Add Condition.



13. Click on Work Item Field Condition.

14. Select Field: Status, Condition: Equal, Value: Done.

15. Click on Next.



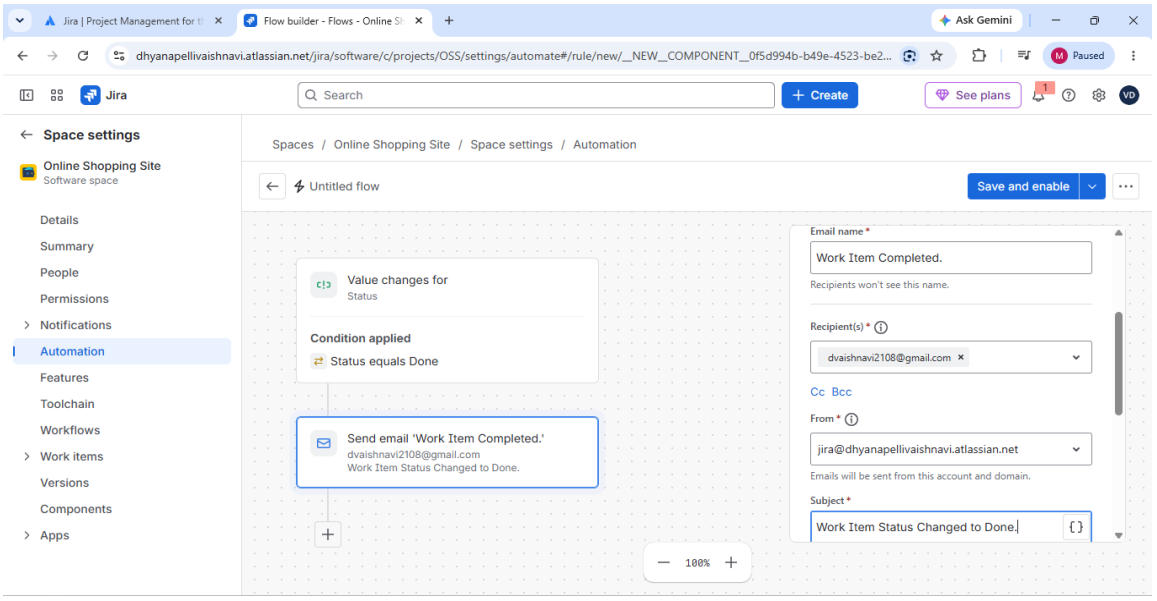
16. Click on Action.

17. Search and select Send Custom Email.

18. Enter the email name as Work Item Completed.

19. Add the recipients and enter the subject as Work Item Status Changed to Done.

20. Click on Next.



b) Configure Automation Rule

Steps:

1. Click on Save and Enable.

2. Enter the Flow Name as Task Completed Email Notification.

3. Set Who can Edit as Private.

4. Click on Turn On Flow.

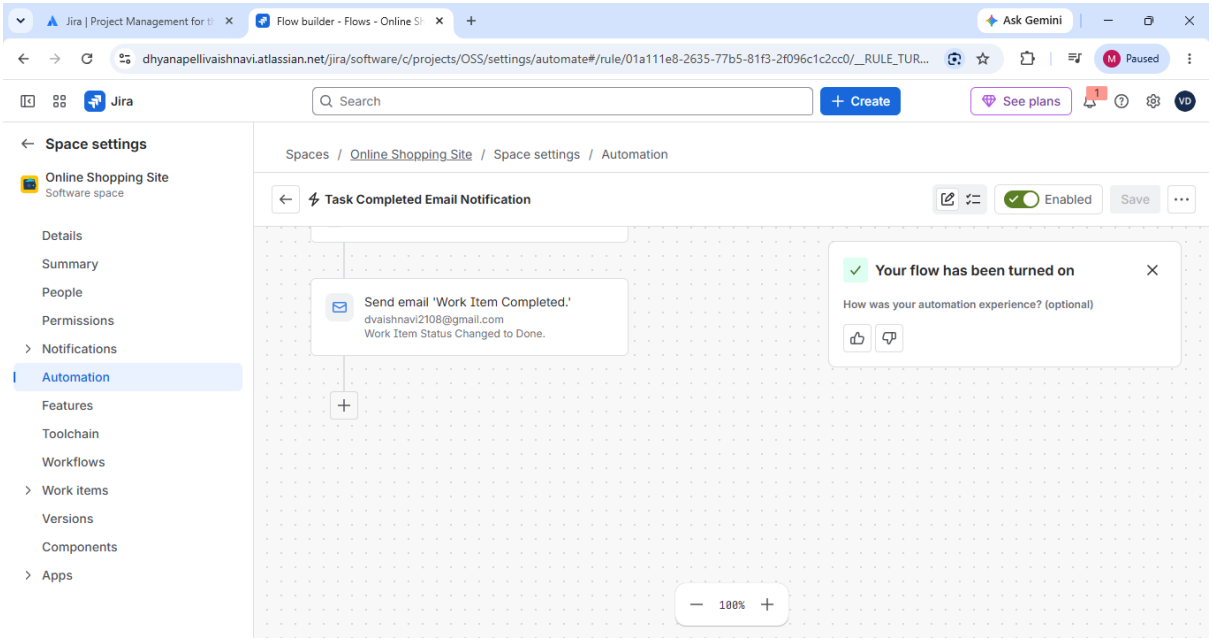
5. Set the Content Format as Plain Text.

6. Enter the following content:

   The work item has been completed successfully. The status of the work item is done now.

   Click on Next.

7. Click on Save and Enable.



c) Test and Enable Automation Rule

Steps:

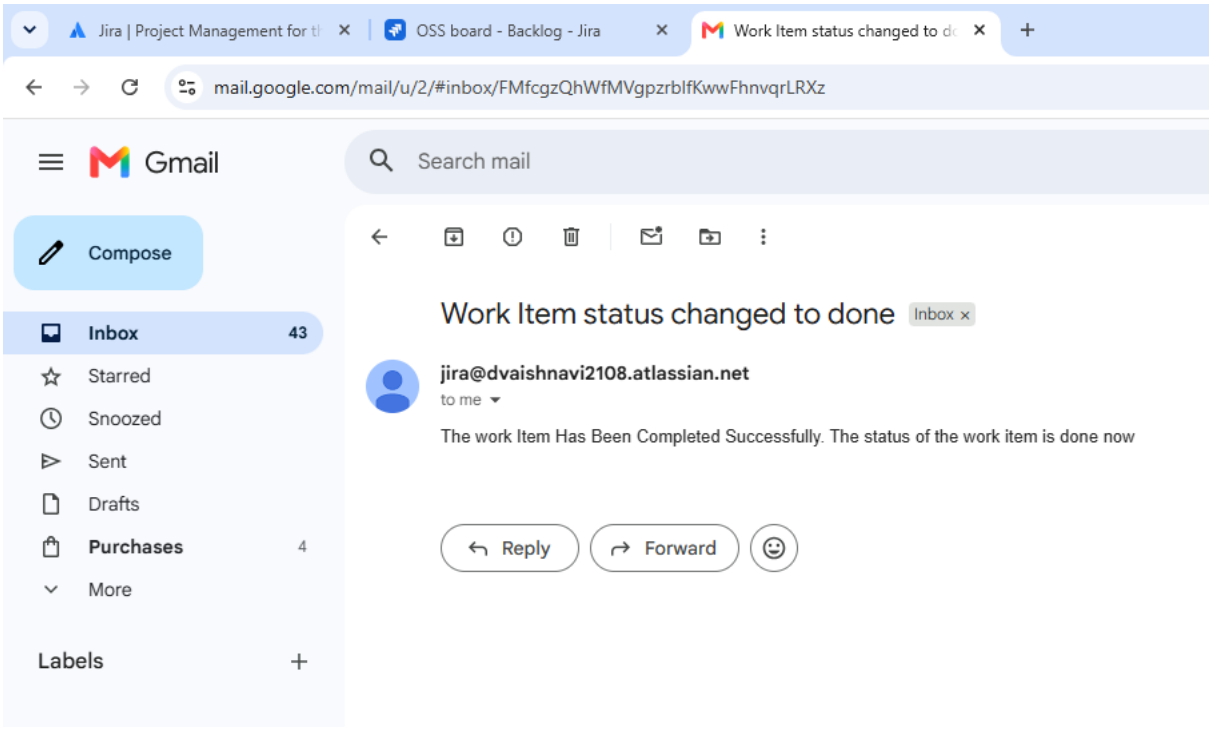
1. Click on Backlog.

2. Click on Todo.

3. Change Todo to Done.

4. The automation rule will be triggered.

5. You will receive an email notification.



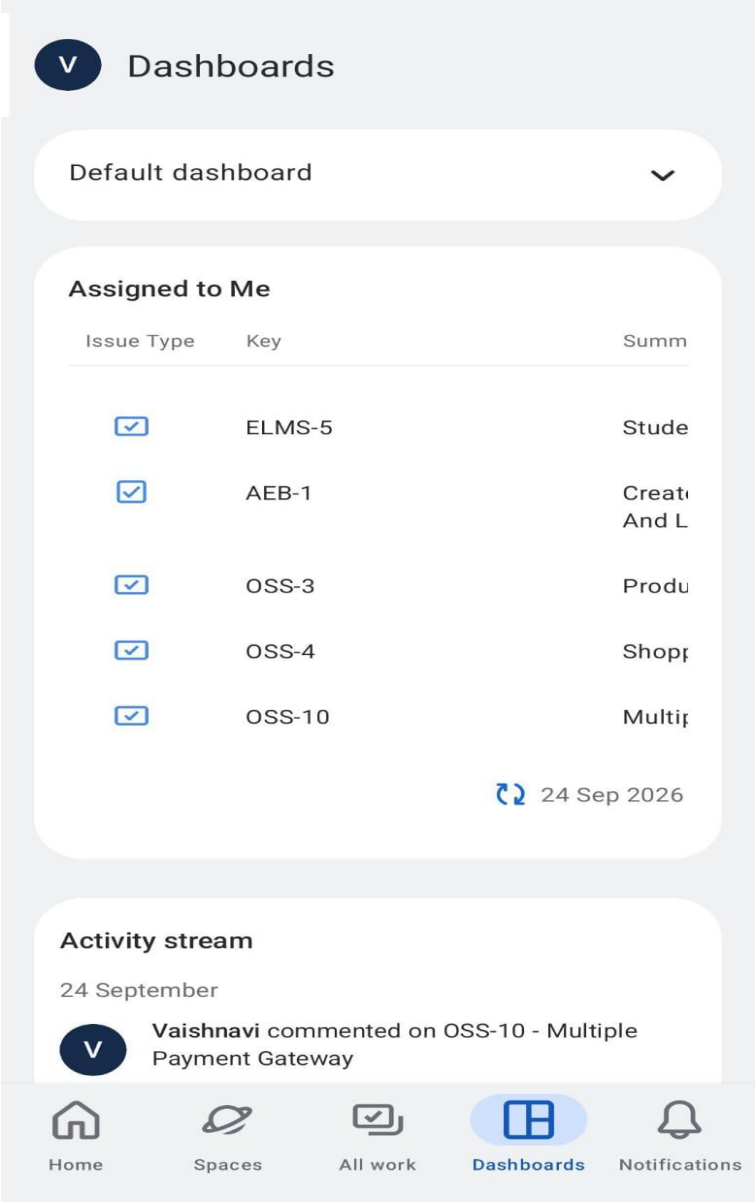
Practical No. 12

Aim:- Introduction to Jira Mobile App.

a) Install and Configure the Jira Mobile App

Steps:

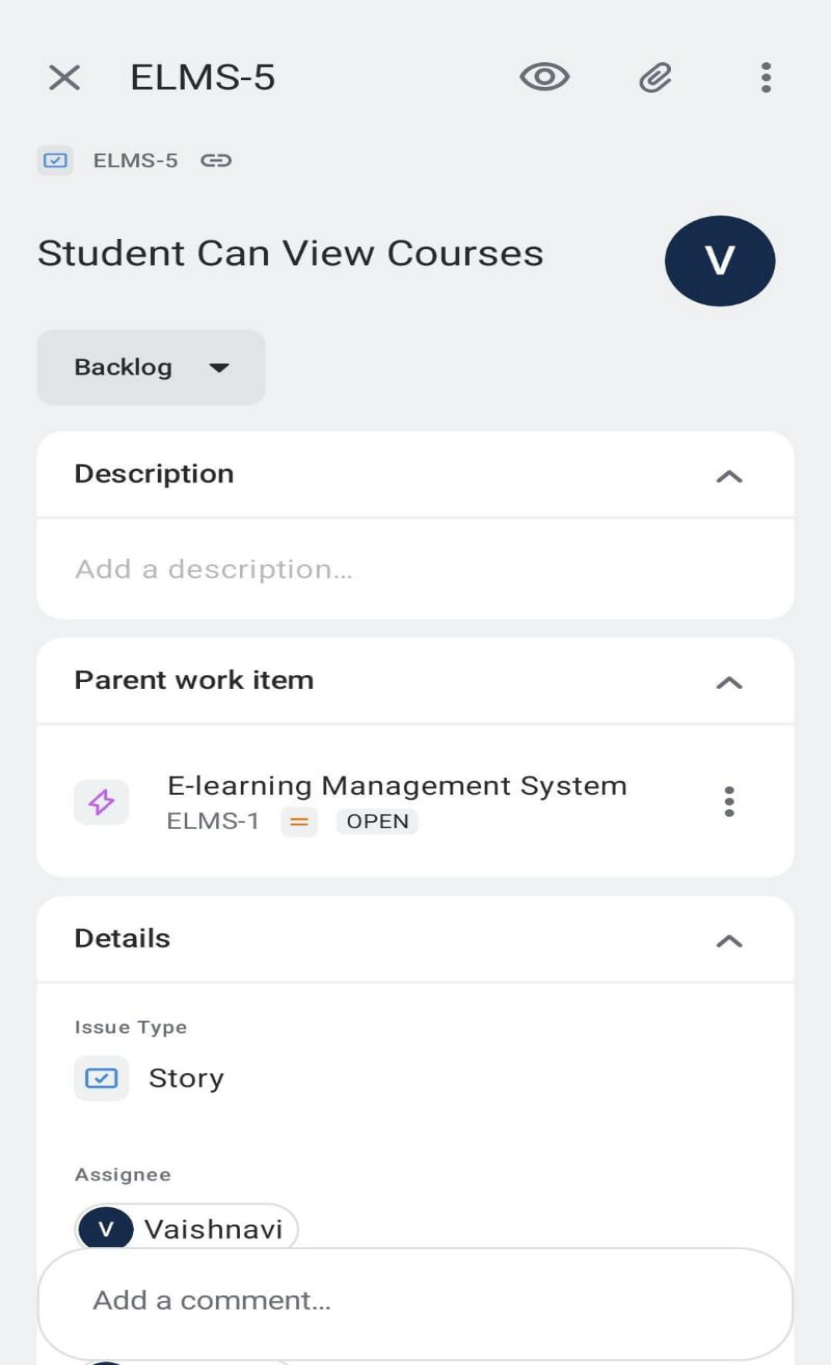
1. Go to Play Store.
2. Search Jira Cloud by Atlassian.
3. Download and login.
4. Set up Jira and select Jira Open → ELMS Board



b) View and Update Issues from Mobile (Status, Comment, Assign).

Steps:

1. Go to Backlog → select In-Progress.
2. Go to Details → check Assignee.
3. Add a Comment → Student can view Courses.



c) Receive Notifications and Manage Real-Time Updates

Steps:

1. Open the Jira mobile app.
2. Go to Notifications and check the received updates.
3. Open the notification to view the latest issue update.

